LinUDP Library

DLL Integration

Version: 2.1.1 (en), 17/06/21

Minimum-firmware release 6.10 Build20210521
Minimum .NET V4.0

Following software examples are tested:
Microsoft Excel 2013 / Windows 7 32 & 64 bit / Windows 8
LabView 2011
Microsoft Visual Studio 2015

© 2021 NTI AG
This work is protected by copyright.
Under the copyright laws, this publication may not be reproduced or transmitted in any form, electronic or mechanical, including photocopying, recording, microfilm, storing in an information retrieval system, not even for didactical use, or translating, in whole or in part, without the prior written consent of NTI AG.

LinMot® is a registered trademark of NTI AG.

Note
The information in this documentation reflects the stage of development at the time of press and is therefore without obligation. NTI AG reserves itself the right to make changes at any time and without notice to reflect further technical advance or product improvement.

NTI AG
LinMot®
Bodenaeckerstr. 2
CH-8957 Spreitenbach

Tel.: +41 (0)56 419 91 91
Fax: +41 (0)56 419 91 92
Email: office@LinMot.com
Homepage: www.LinMot.com

Version 2.1.1

Content

Content ... 3
Use of this document ... 4
Recommended documentation ... 4
Release Note ... 5
1. Overview LinUDP ... 6
1.1. Timing of LinUDP DLL ... 6
1.2 Working principle of LinUDP DLL ... 7
1.3 Installation of the DLL ... 8
1.4 Using LinUDP with Microsoft Excel ... 9
1.5 Using of LinUDP with LabVIEW ... 9
2 Running LinUDP by using the LinMot DLL ... 10
2.1 Minimum steps to operate a LinMot drive with the library ... 11
2.1.1 Axis Communication Interface ... 12
2.1.2 Axis control by using ACI methods ... 13
2.1.3 State Machine handling with ACI functions ... 14
2.2 Overview axis communication interface (ACI) methods and functions (Class ACI) ... 15
2.2.1 ACI specific methods, functions and data access ... 15
2.2.2 State machine related functions, methods and data access ... 16
2.2.3 Motion commands, parameter access ... 19
2.2.4 Curve access ... 21
2.2.5 Command Table access ... 21
2.2.6 Drive parameter access ... 21
2.2.7 Load / save drive configuration ... 21
2.3.0 Details ... 22
2.3.1 ACI specific methods, functions and data access in detail ... 22
2.3.2 State Machine related functions ... 27
2.3.3 Motion commands, parameter access in detail ... 39
2.2.4 Return values during parameter access ... 48
2.3.5 Curve access ... 49
2.3.6 Command Table access ... 52
2.3.7 Drive parameter access ... 53
2.3.8 Load / save drive configuration ... 54
Appendix I Commissioning LinUDP interface ... 57
Appendix II Realtime connection ... 58
Appendix III. LinUDP Testtool ... 59
Appendix IV. Time stamp integration ... 60
Appendix V. Manual Installation of LinUDP DLL driver ... 61
VI.I DLL Installation for Windows 7 ... 62
V.II. DLL installation for Windows XP or older ... 72
Contact ... 76

Use of this document

The presented DLL provides function blocks to control LinMot drives over LinUDP interface.
The library is provided by NTI AG / LinMot free of charge with no warranty for updates.
Also, LinMot accepts no liability for damages that may be caused by using these library.

This document is used for

Controller: C1250-LU(-XX), E1250-LU(-XX), E1450-LU(-XX)

Classification:
[ ] LinMot internally
[ ] Dissemination to customers allowed
[x] Application note

Approval:
[x] Programming example
[ ] Use in productive environments

Recommended documentation

The user manuals are included in LinMot-Talk or can be downloaded on www.linmot.com in the category „Download -> Software & Manuals.

The most important and recommended documents regarding the examples in this documents are shown below:

1. LinMot-Talk
2. Motion Control Software
3. Installation guide C1200 servo controllers
4. Installation guide E1200 servo controllers
5. Installation guide E1400 servo controllers
6. TF Force Control

Release Note

DLL Version	Description
2.0.0	Initial version for use with LinUDP V2, all commands are compatible to the previous version - DLL V1.0.2
2.0.1	Additional functions for support of Master-Slave application added: ➔ setSwitchOn (direct access for setting/resetting of bit 0 Control Word) ➔ setHoming (direct access for setting/resetting of bit 11 Control Word) ➔ isMasterSlaveOperationEnabled (Request, if master and slave axis are operational) MasterSlaveHoming (Similar like „Homing“, here, the homing state of the slaves is observed as well)
2.0.2	Network tools added for internal use
2.0.3	LMmt_WriteLivePar – Correction of data order
2.0.4	Cycle time of communication task improved Special functions with process data combined with monitoring channel for drive timestamp implemented “Pseudo” oscilloscope implemented
2.0.5	LMmt_GenericMC reworked for easy use with all required data types LMav_MoveBestehornRelative added LMav_MoveSinRelative added DLL adapt drives control word on first start Start/Stop/Reboot function reworked (prevention of boot loops, error detection) Helper functions for data type conversion added
2.0.6	Function for setting Error Acknowledge Bit added Additional functions for setting Control Word bit 0 to 15 added Additional function for evaluating the state machine status
2.0.7	New force control functions added: - VAI Go To Pos From Act Pos And Reset Force Control Set I (386xh) - VAI Increment Act Pos And Reset Force Control Set I (387xh) - VAI Inc Act Pos With Higher Force Ctrl Limit and Target Force (388xh)
2.0.8	Deceleration issue on VAI Stop function call fixed. Deceleration was 10 times tool low.
2.1.0	➔ New function for loading curves added ➔ Added new function to load and save the Command Table ➔ Added new functions for saving and restoring a drive configuration
2.1.1	➔ Communication behavior adjusted, as LT 6.10 provides a bug fix on LinUPD V2

1. Overview LinUDP

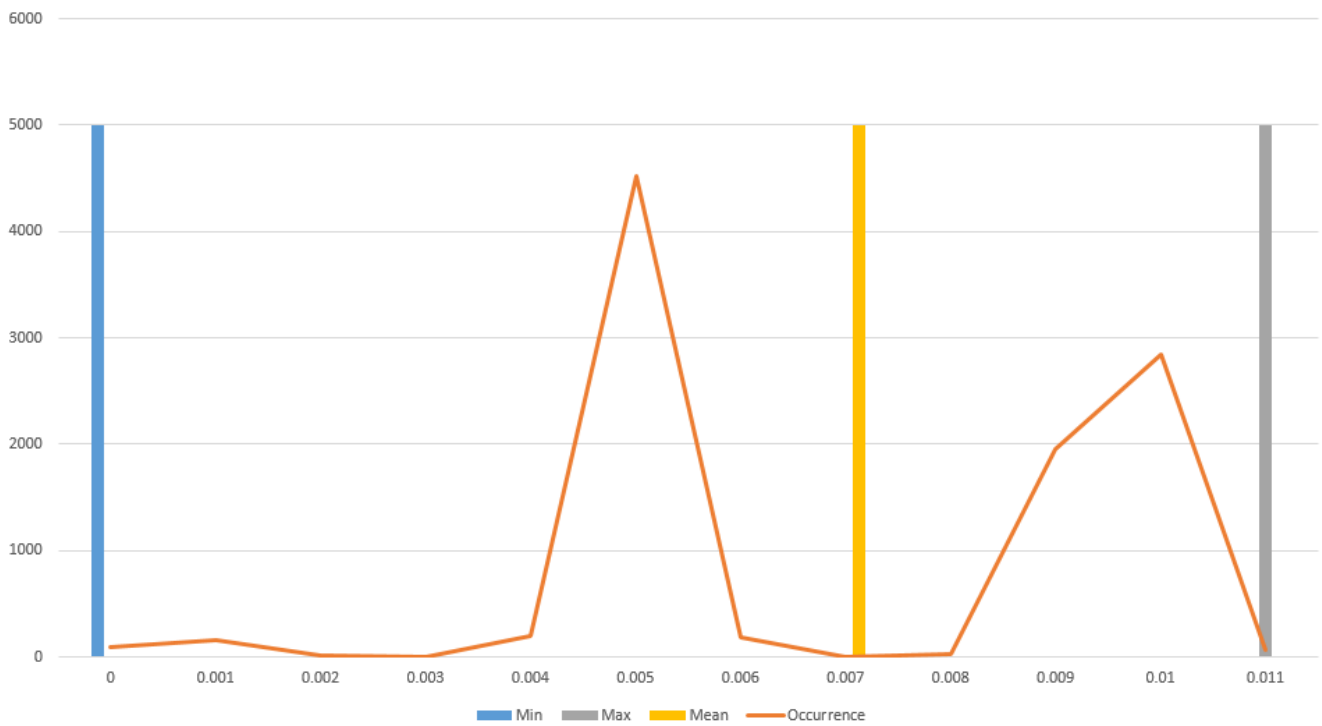
The LinUDP protocol is an easy way for communication with a LinMot servo drive over Ethernet. Here a UDP connection is used. Based on UDP, there are no checks done to make sure if the messages have reached their destination and if they are correctly received. When communicating via LinUDP, the drive has no active function, it only responds to requests with the appropriate answer. The LinUDP protocol is using fixed ports, which can be modified with LinMot Talk. For the master (PC) 41136 is used, for the slaves (LinMot drives) port number 49360. For proper communication you need to configure your firewall port settings if a firewall is active.

1.1. Timing of LinUDP DLL

LinUDP is no real time protocol! The DLL and the user application depend on the system timing controlled by the operating system! Therefore, changes in communication timing and the command execution may occur. This must be handled by application design. There is no possibility to change operation system behavior!

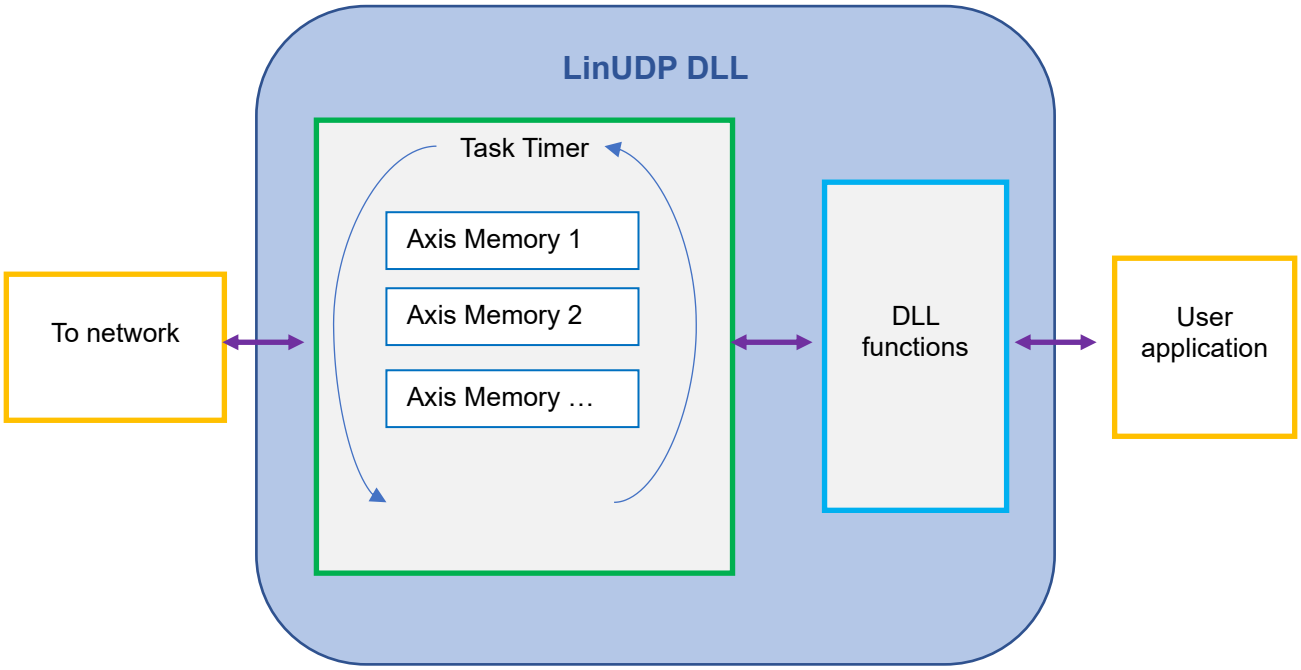
Attention: This DLL can only be used with servo controllers of type „X1X50-LU-XX“! If Ethernet/IP controller are used, LinUDP V2 is not available! Please use the latest DLL of version 1!

Typical data transmission timings tested on Dell LAttitude E5570 with Windows 7 64bit:



This diagram shows the counted data packets to their transmission times (Y-axis packet count, X-axis transmission timing).
During this measurement, 49.86% of data telegrams were handled less than 5ms, but 50.74% above 5ms cycle time. The real cycle time is varying randomly, so no deterministic data exchange is possible. Readings of position or current data versus time are not recommended. During high speed movements, data reading time may be too low for graphical plots!

1.2 Working principle of LinUDP DLL



The DLL reserves data memory for each drive registered with the DLL, in which the incoming and outgoing data from the network are stored (Axis Memory...). The exchange of UDP packets between the DLL and the individual drives on the network is handled by the Task Timer. The actual DLL functions, which are made available from the DLL to the user application, only access the data memory to read data or to set certain functions in the form of motion commands.
This results in the temporal behavior of the communication, since the DLL is executed as a task in Windows and the computing time is controlled by Windows.

1.3 Installation of the DLL

If you only use LabVIEW, you can use the DLL directly without any installation. The DLL file is included in the LabVIEW example folder.
You must install the DLL, if you want to use for example Visual Studio or Excel.
The installation software will run only on computer with Windows 7 or higher! If you still want to use XP, the DLL must be installed manually. Please refer to appendix VI.

For installing the DLL you must have administrator rights. Launch the setup program in the Setup folder and follow the instructions on the screen.
For uninstalling the DLL, open your system manager and choose „Software“. In this list of installed software you can select the „LinUDP Interface Test” and choose “uninstall”.

The installation software register the DLL accordingly to Microsoft .NET guideline into your Windows system. In addition, a small test software will be installed. With this test software you can perform a simple check of the drive connection and its configuration. Details to this test software can be found in appendix V.

Note: The firewall may have to be configured manually to enable the two ports for the DLL

1.4 Using LinUDP with Microsoft Excel

For use of LinUDP with Microsoft Excel, the DLL driver must be installed (see chapter 1.1).
In the \Examples\Excel\ folder you can find an example project for Microsoft Excel. Detailed functions of the DLL can be found in chapter 2. A short instruction how to include the library in Excel can be found on page 50.

1.5 Using of LinUDP with LabVIEW

The DLL is a .NET assembly. For use with LabVIEW you need to create an additional file, which has to be placed in the folder with the running .exe file. For using LabView.exe the file has to be placed in the .exe folder with the file name LabView.exe.conf. If you build an executable file, this file needs to be renamed to Filename.exe.conf (hint: some other LabView versions use .config as file ending!). The file contains the following lines:

```
<?xml version ="1.0"?>
<configuration>
<startup useLegacyV2RuntimeActivationPolicy="true">
<supportedRuntime version="v4.0.30319"/>
</startup>
</configuration>
```

Note: This is necessary to get the DLL running in LabVIEW 2011, because this version use .NET 2.0 as default! This additional file may be different with higher versions of LabVIEW. You can find additional information of this topic at homepage of National Instruments.

For using the DLL functions take a look in the example project!

In the \Examples\LabView\ folder you can find an example for LabVIEW. Detailed functions of the DLL can be found in chapter 2.

2 Running LinUDP by using the LinMot DLL

The following chapters describe the minimum handling of the LinUDP DLL file to get LinUDP running. You need to follow the first chapter for installing the DLL in your Windows system.
If LinUDP is installed you can start using the library functions in your application.

Overview of the used data types:

Type	Type	Type	Type
Boolean	Boolean	Depends on implementation	True or False
Byte	Byte	1 Byte	0 to 255 (unsigned)
Double	Double (real value with double accuracy)	8 Bytes	-1,79769313486231570E+308 to -4,94065645841246544E-324 negative values 4,94065645841246544E-324 to 1,79769313486231570E+308 for positive values
Int32	Whole number	4 Bytes	-2.147.483.648 to 2.147.483.647 (signed)
Int64	Long (long whole number)	8 Bytes	-9.223.372.036.854.775.808 to 9.223.372.036.854.775.807
SByte	SByte	1 Byte	-128 to 127
Int16	Short (short whole number)	2 Bytes	-32.768 to 32.767
Single	Single (floating point value with single precision)	4 Bytes	-3,4028235E+38 to -1,401298E-45 for negative values 1,401298E-45 to 3,4028235E+38 for positive values
String	String (variable length)	Depends on implementation	0 up to 2 billion Unicode-characters
UInt32	UInteger	4 Bytes	0 to 4.294.967.295
UInt64	ULong	8 Bytes	0 to 18.446.744.073.709.551.615
UInt16	UShort	2 Bytes	0 to 65.535

2.1 Minimum steps to operate a LinMot drive with the library

First you need to create an “Axis” object. This object is called “Axis Communication Interface” ACI. The object delivers all defined functions and methods to handle LinMot Drives with LinUDP. The core functions are the settings for the drives IP addresses and ports, called TargetIP and TargetPort, the function to establish and close the UDP connection and the basic functions for drive control (e.g. Control Word handling, Status Word decoding).
Some of the object based functions are split in sub functions to get the library running in different target applications. Based on this, you need to handle the first steps to establish the connection like this:

Declare your object (for example ACI) from LinUDP.ACI
Call the method ClearTargetAddressList().
Then set all IP addresses of the drives you want to access (you need to call the function for every drive!)

ACI.SetTargetAddressList(TargetIP, TargetPort)

where the IP is handled in this format “xxx.xxx.xxx.xxx” and the port “49360” (default).

After you have registered all your drive IP`s and optional port numbers, you can call “ACI.ActivateConnection(“”, “”) where here you must enter your host IP and port (optional, if empty 41136 is used) between the colons.

This steps should be done in a separate program block, which you can start manually, or by start up of your application, if you set your IP addresses hard coded! If you call ACI.ActivateConnection(“”, “”) a second time without closing it before, you get a runtime error. You can use the UDP port only one time in one application!

If you have done this, your ACI object is now working to communicate with the registered drives until you call ACI.CloseConnection(). After you have called this function, the connection is closed. Any other function calls of the ACI object can cause runtime errors!

In VBA it looks like:

```
Dim ACI! As LinUDP.ACI()

Private Sub EstablishConnection()
'Register drives IP address and establish connection
  ACI1.CreateTargetAddressList
  ACI1.ClearTargetAddressList
  ACI1.SetTargetAddressList (IPAddress1, TargetPort)
  ACI1.SetTargetAddressList (IPAddress2, TargetPort)
  X = ACI1.ActivateConnection(HostIP, Port)
End Sub

 Private Sub CloseConnection()
 ' close connection
   X = ACI1.CloseConnection
 End Sub
```

Note: Port number assignment is optional
If you are using the default ports you can send the field entries for TargetPort as empty!

2.1.1 Axis Communication Interface

Before you can use any axis functions you need first a communication link to the Ethernet network. This is necessary, to handle multiple drives. This interface create a communication object, which handles the host port for all connected drives. To get access to the drives, the Axis Communication Interface provides several methods and functions to handle the drives.

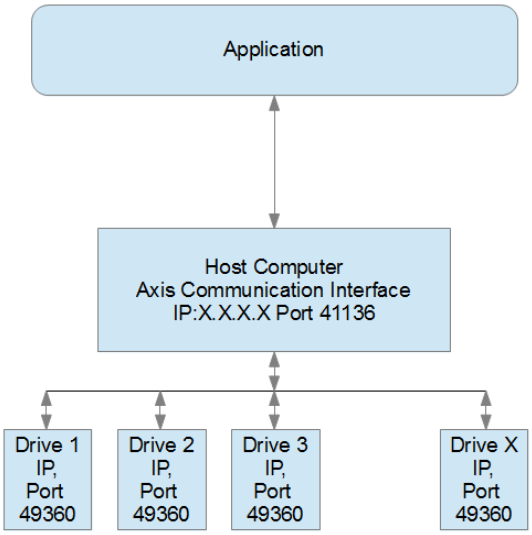


Figure 1: Axis Communication Interface

Declare first an object from the DLL`s object ACI.

In Visual Basic:

```
Public Shared ACI As LinUDP.ACI
```

This object will handle all UDP datagrams to the connected drives. After declaration you need to create the object.

In Visual Basic:

```
ACI = New LinUDP.ACI
```

Then you need to register your drives by transmitting their IP addresses:

```
ACI.ClearTargetAddressList
ACI.SetTargetAddressList (IPAddress1, TargetPort)
ACI.SetTargetAddressList (IPAddress2, TargetPort)
                          .
X = ACI.ActivateConnection("", "")
```

If you want to use a specific IP address (multiple network cards) you have to enter the IP address like „192.1.1.10“. If you don`t use the default port number 41136 you need to enter the configured port number!

2.1.2 Axis control by using ACI methods

For the communication with a drive you need to access the drive with the drive`s IP address.
All drive related functions and methods needs the drive IP to handle the data transmission. The following example shows this handling.

If you want to test, if the specified drive is “Switch On” you will call the object method like:

```
IsActive = ACI.isSwitchOnActive(TargetTP)     / TargetIP e.g.: “192.1.3.5”
```

This function delivers a boolean response to “IsActive”

```
ActPos = ACI.getActualPos(TargetIP)          / TargetIP e.g.: “192.1.3.5”
```

This function delivers a response as double scaled in mm

Sending motion commands:

```
ACI.LMmt_MoveAbs(TargetIP, Pos, MaxVel, Acc, Dec)/ TargetIP e.g.: “192.1.3.5”
```
(Pos, MaxVel, Acc, Decc are single-values)

If you use this Method like:

```
isDone =  ACI.LMmt_MoveAbs(TargetIP, Pos, MaxVel, Acc, Dec)
```

You get a boolean response. True, if the TargetIP is in the drive list which was send during the initialization, false, if the TargetIP don`t exist. The command will be executed based on the defined values.

2.1.3 State Machine handling with ACI functions

All LinMot Drives are using an interface for all basic operations to handle the drive states. This interface is called State Machine (for detailed information see Usermanual_MotionCtrlSW_e_recent.pdf). Figure 24 shows the state machine interface from the ACI object. To get a drive up and running, you need to handle the state machine by your application. The necessary “TargetIP” is not shown in this overview, but it has to be used in the application program.

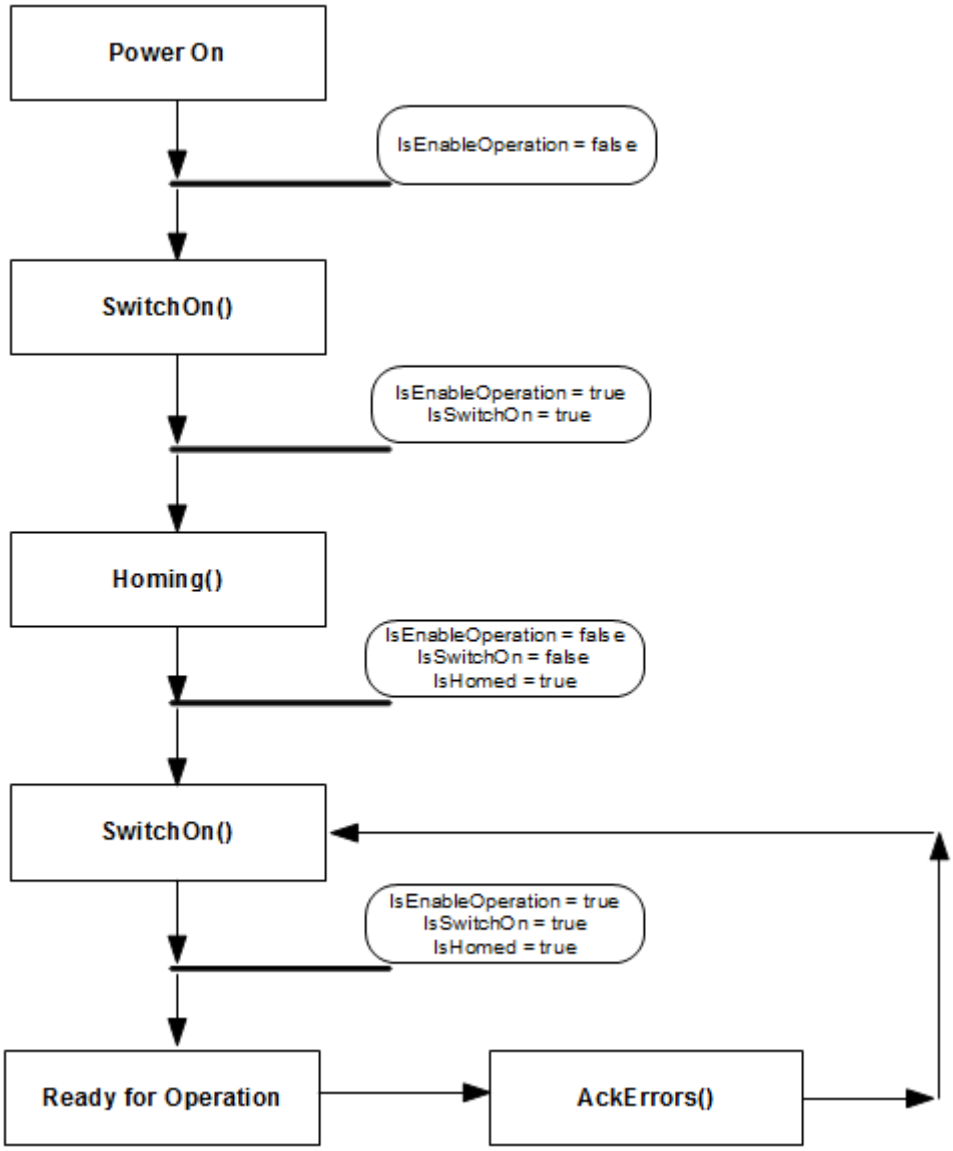


Figure 2: State Machine

Example: The drive is powered up. For the first start you need to switch on the drive. To do this, you use the ACI object with the method SwitchOn and the TargetIP of the drive. You can detect the corresponding state by using the function isSwitchOnActive(TargetIP).
It will look like this:

```
ACI.SwitchOn(TargetIP)
if ACI.isSwitchOnActive(TargetIP) and not ACI.isHomed(TargetIP) then ACI.Homing(TargetIP)
```

Your Application needs to handle the start-up which is shown in figure 2 (see detailed information in the MotionCtr manual).

2.2 Overview axis communication interface (ACI) methods and functions (Class ACI)

For handling LinMot drives you need to declare and create an axis object. This axis object contains several methods, functions and data, which are used to operate the assigned drive. The following overview shows which functions are available. In the following chapters you will find detailed information about these functions.

2.2.1 ACI specific methods, functions and data access

```
New()
boolean getTimerCycle()
boolean SetTimerCycle(Cycle As UInt32)
long getDatagramCycleTime(TargetIP As String)
boolean CreateTargetAddressList() -> Obsolete
boolean ClearTargetAddressList()
boolean SetTargetAddressList(TargetIP As String, TargetPort As String)
boolean ActivateConnection(LocalIP As String, LocalPort As String)
setHostMAC(MAC As String, Port As String)
string getHostIP()

string SetTargetAddressListByMAC(TargetMAC As String, TargetPort As String)
boolean ActivateConnectionByMAC(LocalMAC As String, LocalPort As String)
string getDriveIP_byMAC(MACAdress As String, HostIP As String, HostMetMask As String,
TargetPort As String)
boolean CloseConnection()
Free()
```

DLL Status Information:

```
boolean isConnected(TargetIP As String)
boolean isResponseUpToDate(TargetIP As String)
boolean isRealtimeConfigUpToDate(TargetIP As String)
boolean isNetWorkRunning()
string getDLLError()
boolean clearDLLErrors()
String getVersion()
```

2.2.2 State machine related functions, methods and data access

```
boolean Active(TargetIP As String)
boolean SwitchOn(TargetIP As String)
boolean Homing(TargetIP As String)
boolean AckErrors(TargetIP As String)
boolean JogPlus(TargetIP As String)
boolean JogMinus(TargetIP As String)
boolean isSwitchOnActive(TargetIP As String)
boolean isEventHandlerActive(TargetIP As String)
boolean isSpecialMotionActive(TargetIP As String)
boolean isInTargetPosition(TargetIP As String)
boolean isHomed(TargetIP As String)
boolean isFatalError(TargetIP As String)
boolean isMotionActive(TargetIP As String)
boolean isRangeIndicator1(TargetIP As String)
boolean isRangeIndicator2(TargetIP As String)
boolean isOperationEnable(TargetIP As String)
boolean isEnableOperation(TargetIP As String)
boolean isError(TargetIP As String)
boolean isSafeVoltageEnable(TargetIP As String)
boolean isQuickStop(TargetIP As String)
boolean isSwitchOnLocked(TargetIP As String)
boolean isWarning(TargetIP As String)
boolean isNotReadyToSwitchOnSM(TargetIP As String)
boolean isSwitchOnDisabledSM(TargetIP As String)
boolean isReadyToSwitchOnSM(TargetIP As String)
boolean isSetupErrorSM(TargetIP As String)
boolean isErrorSM(TargetIP As String)
boolean isHardwareTestsSM(TargetIP As String)
boolean isReadyToOperateSM(TargetIP As String)
boolean isOperationEnabledSM(TargetIP As String)
boolean isHomingSM(TargetIP As String)
Boolean setHomingBit(TargetIP As String, State As Boolean)
Boolean setSwitchOnBit(TargetIP As String, State As Boolean)
Boolean setErrorAcknowledgeBit(TargetIP As String, State As Boolean)

Boolean setBit0(TargetIP As String, State As Boolean)
Boolean setBit1(TargetIP As String, State As Boolean)
Boolean setBit2(TargetIP As String, State As Boolean)
Boolean setBit3(TargetIP As String, State As Boolean)
Boolean setBit4(TargetIP As String, State As Boolean)
Boolean setBit5(TargetIP As String, State As Boolean)
Boolean setBit6(TargetIP As String, State As Boolean)
Boolean setBit7(TargetIP As String, State As Boolean)
Boolean setBit8(TargetIP As String, State As Boolean)
Boolean setBit9(TargetIP As String, State As Boolean)
Boolean setBit10(TargetIP As String, State As Boolean)
Boolean setBit11(TargetIP As String, State As Boolean)
Boolean setBit12(TargetIP As String, State As Boolean)
Boolean setBit13(TargetIP As String, State As Boolean)
Boolean setBit14(TargetIP As String, State As Boolean)
Boolean setBit15(TargetIP As String, State As Boolean)

Boolean setJogPlus(TargetIP As String, State As Boolean)
Boolean setJogMinus(TargetIP As String, State As Boolean)
```

```
double getActualPos(TargetIP As String)
double getCurrent(TargetIP As String)
double getDemandPos(TargetIP As String)
long getMonitoringChannel1(TargetIP As String)
long getMonitoringChannel2(TargetIP As String)
long getMonitoringChannel3(TargetIP As String)
long getMonitoringChannel4(TargetIP As String)

Boolean isMasterSlaveOperationEnabled(TargetIP As String)
Boolean MasterSlaveHoming(TargetIP As String)

StateMachineStates getStateMAchineState(TargetIP As String, State As Boolean)


TimeStampData getActualPosWithTimestamp(TargetIP As String)
TimeStampData getCurrentWithTimestamp (TargetIP As String)
TimeStampData getDemandPosWithTimestamp (TargetIP As String)
TimeStampMonitoring getMonitoringChannelWithTimestamp (TargetIP As String, Channel As
Integer)

TimeStampDataUTC getActualPosWithTimestampUTC(TargetIP As String)
TimeStampDataUTC getCurrentWithTimestampUTC(TargetIP As String)
TimeStampDataUTC getDemandPosWithTimestampUTC(TargetIP As String)
TimeStampMonitoringUTC getMonitoringChannelWithTimestampUTC(TargetIP As String, Channel As
Integer)

Integer PseudoScopeSamples()
Boolean EnablePseudoScopeTrace(TargetIP As String)
Boolean isPseudoScopeSampling()
List(of PseudoScopeEntry) getPseudoScopeSamples(TargetIP As String)
Integer getPseudoScopeProgress(TargetIP As String)
```

Helper functions

```
UInt32 LongRawDataToUInt32(value As long)
Int32 LongRawDataToInt32(value As long)
UInt16 LongRawDataToUInt16(value As long)
Int16 LongRawDataToInt16(value As long)
```

Enumerations

```
Enum StateMachineStates asUShort
        NotReadyToSwitchOn = 0
        SwitchOnDisabled = 1
        ReadyToSwitchOn = 2
        SetupError = 3
        GeneralError = 4
        HWTests = 5
        ReadyToOperate = 6
        OperationEnabled = 8
        Homing = 9
        HomingFinished = 90
        ClearanceCheck = 10
        ClearanceCheckFinished = 100
        GoingToInitialPosition = 11
        GoingToInitialPositionFinished = 110
        Aborting = 12
        Freezing = 13
        QuickStopOnError = 14
        GoingToPosition = 15
        GoingToPositionFinished = 150
        JoggingPlus = 16
        Jogging_MovingPositive = 161
        JoggingPlusFinished = 162
        JoggingMinus = 17
        Jogging_MovingNegative = 171
        JoggingMinusFinished = 172
        Linearizing = 18
        PhaseSearch = 19
        SpecialMode = 20
        BrakeDelay = 21
    End Enum
```

2.2.3 Motion commands, parameter access

```
boolean LMmt_MoveAbs(TargetIP As String, Pos1 As Single, MaxVel1 As Single, Acc1 As
Single, Dec1 As      Single)
boolean LMmt_MoveRel(TargetIP As String, Pos1 As Single, MaxVel1 As Single, Acc1 As
Single, Dec1 As                        Single)
boolean LMmt_StartCTCommand(TargetIP As String, CTEntryID As UInteger)
boolean LMmt_ClearEventEvaluation(TargetIP As String)
boolean LMmt_Stop(TargetIP As String, Decceleration As Single)
boolean LMmt_WriteLivePar(TargetIP As String, UPID As UInteger, UPIDValue As Single)
boolean LMav_Mod16BitCTPar(TargetIP As String, CTEntryID As Integer, ParaOffset As
Integer, ParaValue As Integer)
boolean LMav_Mod32BitCTPar(TargetIP As String, CTEntryID As Integer, ParaOffset As
Integer, ParaValue As Integer)
boolean LMav_RunCurve(TargetIP As String, CurveID As Single, CurveOffset As Single,
TimeScale As Integer, AmplitudeScale As Integer)
boolean LMav_MoveBestehorn(TargetIP As String, Position As Single, Velocity As Single,
Acceleration As Single, Jerk As Single)
boolean LMav_MoveBestehornRelative(TargetIP As String, Position As Single, Velocity As
Single, Acceleration As Single, Jerk As Single)
boolean LMav_MoveSin(TargetIP As String, Position As Single, Velocity As Single,
Acceleration As Single)
boolean LMav_MoveSinRelative(TargetIP As String, Position As Single, Velocity As Single,
Acceleration As Single)
boolean LMfc_ChangeTargetForce(TargetIP As String, TargetForce As Single)
boolean LMfc_GoToPosForceCtrlHighLim(TargetIP As String, Position As Single, Velocity As
Single,      Acceleration As Single, ForceLimit As Single, TargetForce As Single)
boolean LMfc_GoToPosForceCtrlLowLim(TargetIP As String, Position As Single, Velocity As
Single,      Acceleration As Single, ForceLimit As Single, TargetForce As Single)
boolean LMfc_GoToPosRstForceCtrl(TargetIP As String, Position As Single, Velocity As
Single, Acceleration As Single, Deceleration As Single)
boolean LMfc_GoToPosRstForceCtrlSetI(TargetIP As String, Position As Single, Velocity As
Single, Acceleration As Single, Deceleration As Single)
boolean LMfc_IncrementActPosAndResetForceControlSetI(TargetIP As String, Position As Single,
Velocity As Single, Acceleration As Single, Deceleration As Single)
boolean LMfc_IncrementActPosWithHigherForceCtrlLimitAndTargetForce(TargetIP As String,
PositionIncrement As Single, MaxVelocity As Single, Acceleration As Single, ForceLimit As
Single, TargetForce As Single)

string LMcf_GetErrorTxt(TargetIP As String)
integer LMcf_GetErrorCode(TargetIP As String)
string LMcf_GetWarningTxt(TargetIP As String)
integer LMcf_GetWarningCode(TargetIP As String)
long LMcf_StartStopDefault(TargetIP As String, Mode As Integer)
long getROM_ByUPID(TargetIP As String, UPID As Uint32)
long getRAM_ByUPID(TargetIP As String, UPID As Uint32)
long getMinVal_ByUPID(TargetIP As String, UPID As Uint32)
long getMaxVal_ByUPID(TargetIP As String, UPID As Uint32)
long getDefault_ByUPID(TargetIP As String, UPID As Uint32)
long setRAM_ByUPID(TargetIP As String, UPID As Uint32, Value As long)
long setROM_ByUPID(TargetIP As String, UPID As Uint32, Value As long)
long setRAM_ROM_ByUPID(TargetIP As String, UPID As Uint32, Value As long)
boolean LMav_SetCurrentCommandMode(TargetIP As String, Current As Single)
boolean LMav_ResetCurrentCommandMode(TargetIP As String)
```

boolean LMmt_GenericMC(TargetIP As String,Parameters according to table 1)

MCHeader	MCParaWord0	MCParaWord1	MCParaWord2	MCParaWord3	MCParaWord4
UInt16	UInt16				
UInt16	UInt16	Uint16			
UInt16	UInt16	SInt16			
UInt16	UInt16	UInt32			
UInt16	UInt16	SInt32			
UInt16	SInt32	UInt32	UInt32	UInt32	
UInt16	UInt32				
UInt16	UInt32	UInt32	UInt32		
UInt16	SInt32				
UInt16	SInt32	SInt32			
UInt16	SInt32	SInt32	SInt32		
UInt16	UInt16	SInt32	SInt32		
UInt16	UInt16	SInt32	UInt16	SInt16	
UInt16	UInt16	SInt32	UInt32	SInt16	
UInt16	UInt16	UInt16	SInt16		
UInt16	UInt16	UInt16	SInt32		
UInt16	UInt16	UInt16	SInt32	SInt32	
UInt16	UInt16	UInt16	SInt32	SInt32	SInt32
UInt16	UInt16	SInt32	SInt32	SInt32	
UInt16	UInt16	SInt32	SInt16	SInt32	
UInt16	SInt16	UInt16	UInt16	UInt16	
UInt16	SInt16				
UInt16	SInt32	UInt32			
UInt16	SInt32	UInt32	UInt32		
UInt16	SInt32	UInt32	UInt32	UInt16	
UInt16	SInt32	UInt32	UInt32	UInt32	
UInt16	SInt16	UInt16	UInt16		
UInt16	UInt32	UInt32			
UInt16	UInt16	SInt32	SInt32	SInt32	SInt32
UInt16	SInt16	SInt32			
UInt16	SInt32	UInt16			
UInt16	SInt32	UInt16	UInt16		
UInt16	UInt16	UInt16	UInt16	UInt16	
UInt16	SInt32	UInt32	UInt32	SInt16	SInt16
UInt16	SInt32	UInt32	UInt32	UInt32	UInt32

Table 1: Parameter options for use of generic function

2.2.4 Curve access

```
boolean LMcf_LoadCurve(TargetIP As String, CurveID As Integer, SetpointCount As Integer,
CurveName    As String, Xlength As Integer, Xdim As Integer, Ydim As Integer, Mode As
Integer,     Setpoints As Integer())
Boolean LMcf_LoadCurve(TargetIP As String, Mode As Integer, CurveData As
CurveDataDefinition)

boolean LMcf_isCurveLoading(TargetIP As String)
integer LMcf_getCurveProgress(TargetIP As String)
CurveDataDefinition getUploadedCurveData(TargetIP As String)
Boolean LMcf_setDownloadCurveData(TargetIP As String, CurveData As CurveDataDefinition)
Boolean LMcf_StartUploadCurve(TargetIP As String, CurveID As UInt16)
Boolean LMcf_StartDownloadCurve(TargetIP As String)
Integer() LMcf_getAllCurveID(TargetIP As String)
Boolean LMcf_isCurveOnDrive(TargetIP As String, CurveID As UInt16)
Boolean  LMcf_DeleteAllCurvesInRAM(TargetIP As String)
Boolean LMcf_SaveAllCurvesFromRAMToFLASH(TargetIP As String)
```

2.2.5 Command Table access

```
CommandTableStructure LMcf_getCommandTableContent(TargetIP As String)
UInt16 LMcf_DeleteCommandTable_RAM(TargetIP As String)
UInt16 LMcf_WriteCommandTableToFLASH(TargetIP As String)
Boolean LMcf_setCommandTableContent(TargetIP As String, CT As CommandTableStructure)
```

2.2.6 Drive parameter access

```
List(Of UPID_List) LMcf_getUPIDList(TargetIP As String, StartUPID As UShort, StopUPID As
UShort)
List(Of UPID_List) LMcf_getModified_UPIDList(TargetIP As String, StartUPID As UShort,
StopUPID As UShort)
Boolean LMcf_setUPIDList(TargetIP As String, UPIDList As List(Of UPID_List))
```

2.2.7 Load / save drive configuration

```
Boolean Save_CommandTable(TargetIP As String, FilePath As String)
Boolean Save_CommandTable(CT As CommandTableStructure, FilePath As String)
Boolean Load_CommandTable(TargetIP As String, FilePath As String)
CommandTableStructure Load_CommandTable(FilePath As String)

Boolean Save_DriveParameters(TargetIP As String, FilePath As String)
Boolean Save_DriveParameters(Data As List(Of UPID_List), FilePath As String)
Boolean  Load_DriveParameters(TargetIP As String, FilePath As String)
List(Of UPID_List) Load_DriveParameters(FilePAth As String)
Boolean Save_Curves(TargetIP As String, FilePath As String)
Boolean Load_Curves(TargetIP As String, FilePath As String)
Boolean Save_DriveConfiguration(TargetIP As String, FilePath As String)
Boolean Load_DriveConfiguration(TargetIP As String, FilePath As String)
UInt32 getDriveHash(TargetIP As String)
Boolean isDriveConfigurationSame(TargetIP As String, FilePath As String)
```

2.3.0 Details

2.3.1 ACI specific methods, functions and data access in detail

Function call: boolean CreateTargetAddressList()

This command is obsolete and for compatibility to old projects still available! Don't use in new applications.

Function call: boolean ClearTargetAddressList()

This method delivers true!

Description:
This method empty the address list of registered target drives. You should call this method after "CreateTargetAddressList". Any communication must be inactive! If you want to add new controller, please call "CloseConnection" first. Then clear the drive list and set up this list new.

Function call: boolean SetTargetAddressList(TargetIP As String, TargetPort As String)

This method delivers true!

Description:
This method creates a axis module in the ACI object and sets the drives IP and port address. You have to call this method for every drive you want to access. This registration has to be done before you call "ActivateConnection"! If you send an empty string, the drive`s default port will be used (49360)!

Function call: boolean ActivateConnection(LocalIP As String, LocalPort As String)

This method delivers true!

Description:
This method is establishing the data link between the DLL object and the target drives. By calling this method you can assign the local IP address and port as a string, if you have more than one network adapter. If you send an empty string for the address, the first available network adapter will be used! If you send an empty string for the port, the default port will be used (41136)! If you want to use a different port, you must configure this port on every drive! The drive site defined master port must be the same for all drives and must be entered here as "LocalPort"!

Function call: boolean CloseConnection()

This method delivers true!

Description:
This method is closing all UDP connections to the registered drives. You can use this, if you need to register additional drives after you have called “ActivateConnection”. It is recommended to call this function by closing your application to give the allocated resources free for other applications.

Function call: Free()

This function is no longer in use. The function is replaced by “Dispose”. For compatibility of old applications, function call will execute “Dispose”.

Funktionsaufruf: Dispose()

This function should be called, while application is closing. This function will release all system ressources, which were allocated by DLL. If this function is called, the whole axis communication data object will be removed. Ongoing function calls will cause in a «Index out of range» error.

Since the improvement with the new communication stack these functions are no longer in use!
~~Function call: boolean setTimerCycle(Cycle As UInt32)~~

~~This function will deliver true, if a valid time span is accepted. It delivers false, if “Cycle” is <= 0. In this case, the last cycle time will be used.~~

~~Description:~~
~~This function may be used, to change the internal time interval for data exchange. By default, this value is set to 5ms. If data exchange may be handled slower, to prevent huge network load, this value may be increased. The value of „Cycle“ corresponds to the cycle time in ms.~~

~~**Note:** The communication runs with Windows-specific modules and is not real time. The cycle time may be used as a minimum time. In reality this value will change to higher values, depending on different running applications!~~

~~Function call: integer getTimerCycle ()~~

~~This function delivers the actual defined cycle time in ms.~~

Function call: long getDatagramCycleTime(TargetIP as String)

Description:
This function delivers the time span between transmitting a request and receiving the drive response in milliseconds

Function call: string getDLLError()

This function delivers active DLL error messages. If no error is active, an empty string will be returned. The message may change, if more than one error is active.

Function call: boolean clearDLLErrors()

This function call reset the last DLL error. Error messages can be read by function call *getDLLError()*.

Function call: boolean isConnected(TargetIP As String)

This function delivers true, if the addressed servo controller is active (connected).

Description:
This function delivers the data exchange state between ACI object and servo controller. True, if connected and data exchange is running, false when no connection.

Function call: boolean isResponseUpToDate(TargetIP As String)

This function delivers the update state of drive response to the last motion command. True, if the last motion command is executed successfully, false, if the command is still in execution.

Function call: boolean isRealtimeConfigUpToDate(TargetIP As String)

This function delivers the update state of drive response to the last parameter channel command. True, if the last parameter channel command is executed successfully, false, if the command is still in execution.

Function call boolean isNetworkRunning()

This command delivers true, if the DLL is in cyclic data exchange. The state can be changed ba using “ActivateConnection” or “CloseConnection”. This function checks only computer side communication. If messages will be sent somewhere, it is still possible to get true!

Function call: String getVersion()

Get the DLL version number which is used.

Special option, if MAC addressing should be used (e.g. IP´s are changing by DHCP).

Attention: To identify a possible LinMot drive, DLL is sending a request to every possible IP address. This address will be calculated based on the computers subnet mask. Depending on the subnet mask, this method bring up huge traffic and may need some time! Use carefully and only, if necessary!

Function call: setHostMAC(MAC As String, Port As String)

With this function you define the network adapter in your computer. The MAC address is defined by card manufacture. The port can be set independent. If the port is empty, the default port 41136 is used. This function must be called first, to define the network adapter which should be used with LinUDP.

Function call: string getHostIP()

This function delivers the actual IP address of the network card, which is selected by „setHostMAC".

Function call: string SetTargetAddressListByMAC(TargetMAC As String, TargetPort As String)

The return value is the IP address of the drive!

This function defines the used controller by its MAC address. This address is printed on the controller label. You can define the "TargetPort" as needed. If this entry is empty, the default port 49360 is used.
The function delivers the corresponding IP address of the controller. This address must be stored for later use with the DLL function calls!

Function call: boolean ActivateConnectionByMAC(LocalMAC As String, LocalPort As String)

The return value is true!

Description:
This method is establishing the data link between the DLL object and the target drives. By calling this method you must assign the MAC address of your network card and port as a string. If you send an empty string for the port, the default port will be used (41136)! If you want to use a different port, you must configure this port on every drive! The drive site defined master port must be the same for all drives and must be entered here as "LocalPort"!

Attention: MAC address must be entered in this form „XX-XX-XX-XX-XX"!

Function call: string getDriveIP_byMAC(MACAdress As String, HostIP As String, HostMetMask As String, TargetPort As String)

This function determines the IP of the drive via its MAC.
The IP and the subnet mask of the PC network card must be specified.
The TargetPort is internally set to 49360 if no specification is made (factory setting of the drive).

Attention: This process can take several minutes. The bigger the address range is defined (subnet-mask) the longer the search process will take!

After a successful search run, the list of found participants is fixed internally for 1 minute. Newly connected subscribers are therefore recognized at the earliest 1 minute after they are connected.

Attention: This function only works if no connection to any drive is established by the DLL!

The state of ***isNetworkRunning()*** must report ***False***, otherwise the return value is always empty!

Attention: MAC address must be entered in this form „XX-XX-XX-XX-XX“!

2.3.2 State Machine related functions

Function call: boolean isSwitchOnActive(TargetIP As String)

This function test the drive, if it is switch on active, the function delivers true.

Function call: boolean isEventHandlerActive(TargetIP As String)

This function test the drive, if an event handler is active, the function delivers true. This may be the case, if a command table sequence is running. Here, you can check, if the command table has finished, to prevent any issues by sending motion commands while table is running.

Function call: boolean isSpecialMotionActive(TargetIP As String)

This function test the drive, if a special motion is active, the function delivers true. (e.g. Current Command Mode).

Function call: boolean isInTargetPosition(TargetIP As String)

This function test the drive, if the stator is in target position, the function delivers true.

Function call: boolean isHomed(TargetIP As String)

This function test the drive, if the drive is homed, the function delivers true.

Function call: boolean isFatalError(TargetIP As String)

This function test the drive, if a fatal error has appeared, the function delivers true. Fatal errors cannot be acknowledged! These errors can be reset by a reboot of the drive.

Function call: boolean isMotionActive(TargetIP As String)

This function test the drive, if a motion is active, the function delivers true.

Function call: boolean isNotReadyToSwitchOnSM(TargetIP As String)

This function delivers true, it the controller is not ready to switch on. To change this state of the state machine, you must toggle the switch on bit 0 by using the command “SwitchOn”. This handling prevents any undefined startup of the controller (see Motion Control Software Manual).

Function call: boolean isSwitchOnDisabledSM(TargetIP As String)

This bit is true, if bit 0 or bit 1 in the Control Word is zero (Monitoring of switch on and SVE/STO or Voltage Enable inputs, depends on the controller).

Function call: boolean isReadyToSwitchOnSM(TargetIP As String)

This bit i true, if the controller is ready to be switched on with „SwitchOn“.

Function call: boolean isSetupErrorSM(TargetIP As String)

This bit is true, if there is any configuration error.

Function call: boolean isErrorSM(TargetIP As String)

This bit is true, if an error occurred.

Function call: boolean isHWTestsSM(TargetIP As String)

During switch on of the drive power stages, the controller performs a self-test of power stage, cables and motor (depend on the controller type). During this phase, this bit is true.

Function call: boolean isReadyToOperateSM(TargetIP As String)

This bit shows “Ready to operate”.

Function call: boolean isOperationEnabledSM(TargetIP As String)

If this bit is true, the controller is in operation and perform any motion command, which will be sent.

Function call: boolean isHomingSM(TargetIP As String)

This bit is true, while a homing procedure is running.

Function call: boolean isRangeIndicator1(TargetIP As String)

This function test the drive, if the range indicator 1 is active, the function delivers true.

Function call: boolean isRangeIndicator2(TargetIP As String)

This function test the drive, if the range indicator 2 is active, the function delivers true.

Function call: boolean isOperationEnable(TargetIP As String)

This function test the drive, if operation enable is active, the function delivers true.

Function call: boolean isEnableOperation(TargetIP As String)

This function test the drive, if enable operation is set, the function delivers true.

Function call: boolean isError(TargetIP As String)

This function test the drive, if an error occurred, the function delivers true.

Function call: boolean isSafeVoltageEnable(TargetIP As String)

This function test the drive, if the safety voltage enable input (or STO) is set, the function delivers true (only available on controller with SVE or STO functionality).

Function call: boolean isQuickStop(TargetIP As String)

Diese Funktion prüft, ob ein QuickStop ausgelöst ist. QuickStop ist als negative Logik ausgelegt, daher muss dieses Bit gesetzt sein, damit der Regler arbeitet.

Function call: boolean isSwitchOnLocked(TargetIP As String)

This function test the drive, if quickstop is set, the function delivers true. The QuickStop function is in negative logic, so this bit must be true, to get the controller active.

Function call: boolean isWarning(TargetIP As String)

This function test the drive, if a warning occurred, the function delivers true.

Method call: boolean isMasterSlaveOperationEnabled(TargetIP As String)

This method delivers true, if Master and connected Slaves are ready for operation.

HINT: This function will work only, if Master/Slave application is installed on the drives and minimum one slave is connected to the master!

Function call: double getActualPos(TargetIP As String)

This method delivers the actual slider position as a double value, unit is mm.

Function call: double getCurrent(TargetIP As String)

This method delivers the actual motor current as a double value, unit is in ampere.

Function call: double getDemandPos(TargetIP As String)

This method delivers the actual slider demand position as a double value, unit is mm.

Function call: long getMonitoringChannel1(TargetIP As String)

This method delivers the monitoring channel 1 value as long. Depending on the assigned UPID this raw data value must be scaled.

Function call: long getMonitoringChannel2(TargetIP As String)

This method delivers the monitoring channel 2 value as long. Depending on the assigned UPID this raw data value must be scaled.

Function call: long getMonitoringChannel3(TargetIP As String)

This method delivers the monitoring channel 3 value as long. Depending on the assigned UPID this raw data value must be scaled.

Function call: long getMonitoringChannel4(TargetIP As String)

This method delivers the monitoring channel 4 value as long. Depending on the assigned UPID this raw data value must be scaled.

Attention: If functions with timestamp are used, monitoring channel 1 on all drives must be configured to UPID 82h (Operating Sub Hours) !

Function call: TimestampData getActualPosWithTimestamp(TargetIP As String)

This function returns the actual position and the time stamp of reading in mm and ms.
This function read out the drive sub hours in ms. This function does not handle the hour counting! Each hour, time value change and start from zero!

Function call: TimestampData getCurrentWithTimestamp (TargetIP As String)

This function returns the actual current and the time stamp of reading in ampere and ms.
This function read out the drive sub hours in ms. This function does not handle the hour counting! Each hour, time value change and start from zero!

Function call: TimestampData getDemandPosWithTimestamp (TargetIP As String)

This function returns the demand position and the time stamp of reading in mm and ms.
This function read out the drive sub hours in ms. This function does not handle the hour counting! Each hour, time value change and start from zero!

Function call: TimestampMonitoring getMonitoringChannelWithTimestamp(TargetIP As String, Channel As Integer)

This function returns the raw data value of the selected monitoring channel (2 – 4). Channel values out of range of 2 – 4 will be treated to channel 2! Raw data value must be scaled/converted according to the definitions in LinMot Talk. In addition, time stamp of reading is included. Monitoring Channel 1 is predefined for «Timestamp» functions and must be configured on every drive!

This function read out the drive sub hours in ms. This function does not handle the hour counting! Each hour, time value change and start from zero!

Used return data structure of timestamp function calls:

```
Structure TimestampData
        value As Double
        Timestamp As Long

Structure TimestampMonitoring
        value As Long
        Timestamp As Long
```

Attention: If functions with timestamp are used, monitoring channel 1 on all drives must be configured to UPID 82h (Operating Sub Hours) !

Attention: If functions with timestamp are used, monitoring channel 1 on all drives must be configured to UPID 82h (Operating Sub Hours) !

Function call: TimestampData getActualPosWithTimestampUTC(TargetIP As String)

This function returns the actual position and the time stamp of reading in mm and ms.
This function read out the drive sub hours in ms. Time value include the hour counting and is delivered as coordinated universal time.

Function call: TimestampData getCurrentWithTimestampUTC(TargetIP As String)

This function returns the actual current and the time stamp of reading in ampere and ms.
This function read out the drive sub hours in ms. Time value include the hour counting and is delivered as coordinated universal time.

Function call: TimestampData getDemandPosWithTimestampUTC(TargetIP As String)

This function returns the demand position and the time stamp of reading in mm and ms.
This function read out the drive sub hours in ms. Time value include the hour counting and is delivered as coordinated universal time.

Function call: TimestampMonitoring getMonitoringChannelWithTimestampUTC(TargetIP As String, Channel As Integer)

This function returns the raw data value of the selected monitoring channel (2 – 4). Channel values out of range of 2 – 4 will be treated to channel 2! Raw data value must be scaled/converted according to the definitions in LinMot Talk. In addition, time stamp of reading is included. Monitoring Channel 1 is predefined for «Timestamp» functions and must be configured on every drive!

This function read out the drive sub hours in ms. Time value include the hour counting and is delivered as coordinated universal time.

Used return data structure of timestamp function calls:

```
Structure TimestampDataUTC
        value As Double
        Timestamp As DateTime

Structure TimestampMonitoringUTC
        value As Long
        Timestamp As DateTime
```

Attention: If functions with timestamp are used, monitoring channel 1 on all drives must be configured to UPID 82h (Operating Sub Hours) !

Helper functions:

Function call: UInteger LongRAWDataToUINT32(value as Long)

This function may be used to convert “Long” data values from monitoring channels to the needed data type «UInt32» (see LinMot Talk data type of the selected variable).

Function call: Integer LongRAWDataToINT32(value as Long)

This function may be used to convert “Long” data values from monitoring channels to the needed data type «Int32» (see LinMot Talk data type of the selected variable).

Function call: UInt16 LongRAWDataToUINT16(value as Long)

This function may be used to convert “Long” data values from monitoring channels to the needed data type «UInt16» (see LinMot Talk data type of the selected variable).

Function call: Int16 LongRAWDataToINT16(value as Long)

This function may be used to convert “Long” data values from monitoring channels to the needed data type «Int16» (see LinMot Talk data type of the selected variable).

PseudoScope feature:
This «PseudoScope» feature is foreseen for simple data acquisition. This data acquisition is not precise like LinMot Talk oscilloscope because of non-deterministic operation system!

Functional description:
This feature provide a service, which will store a defined amount of data readings by «PseudoScopeSamples» in DLL memory.
If the amount of readings is reached, data collection is finished and data is ready for reading out from DLL memory.
Data read out is done as a list of elements, defined as structure «PseudoScopeEntry».
In this structure, every data entry provide the drive timestamp. This may be used for diagrams.

Based on non-deterministic fieldbus, time interval between single data reading may change. It is not possible to achieve a fix time interval between the readings.

If fast signal changes occur, signal changes may be not read!

Attention: If functions with «PseudoScope» are used, monitoring channel 1 on all drives must be configured to UPID 82h (Operating Sub Hours) !

Function call: Integer PseudoScopeSamples()

This function call read or write the amount of data points, which should be recorded.
For each network data exchange, one new data point is stored in memory.
Maximum available amount of data points is not automatically handled! If this value is too big, system memory may be not sufficient for storage!

Attention: for each used drive, this amount of data points will be reserved automatically in memory!

Function call: Boolean EnablePseudoScopeTrace(TargetIP As String)

This function enables data acquisition for the called IP address. Data acquisition starts directly with this function call. The return value corresponds to the status of the recording – true = recording active.

Function call: Boolean isPseudoScopeSampling()

This function call check, if data acquisition is still running or finished. As long as this function delivers true, data acquisition is running. If the defined amount of data points is reached, function call delivers false.

Function call: List(of PseudoScopeEntry) getPseudoScopeSamples(TargetIP As String)

This function call read the stored data from the called IP address out f DLL memory. The data is read out as list of data structure «PseudoScopeEntry».

Used return data structure of «PseudoScopeEntry»

```
Structure PseudoScopeEntry
        ActualPos As Double
        DemandPos As Double
        Current As Double
        TimeStamp As Long
        MonitoringChannel2 As Long
        MonitoringChannel3 As Long
        MonitoringChannel4 As Long
```

This function use the drive sub hours in ms. This function does not handle the hour counting! Each hour, time value change and start from zero!

Function call: Integer getPseudoScopeProgress(TargetIP As String)

This function delivers the data acquisition progress oft he called IP. Progress is delivered in a range of 0 – 100%.

Attention: If functions with «PseudoScope» are used, monitoring channel 1 on all drives must be configured to UPID 82h (Operating Sub Hours) !

Function call: boolean Active(TargetIP As String)

This function is deprecated and should not be used in new applications. The command is available for compatibility to the old DLL.

Function call: boolean SwitchOn(TargetIP As String)

This function delivers true, if bit 0 in the „Control Word“ is set to 1, false, if it is set to 0.

Description:
This function set/reset bit 0 in the „Control Word“ of the specified controller. If the bit on the controller is 0 then it will be set to 1, if it is 1, it will be set to 0. This function call will toggle this bit!

Function call: boolean Homing(TargetIP As String)

The result is true, if the function was successful.

Description:
This function will perform a homing procedure on the specified controller. The response occurs, if the function is executed. If a timeout of 60s is reached, the function will abort and a timeout error message will be created in the DLL error.

Function call: boolean AckErrors(TargetIP As String)

The result is true, if the execution was successful.

Description:
This function will reset pending errors on the specified controller (Fatal errors cannot be reset).

Function call: boolean JogPlus(TargetIP As String)

The result is true, if the execution was successful.

Description:
This function runs the jog mode in positive direction. For stopping this mode, call the function again! The jog mode behavior must be configured on the controller with LinMot Talk!

Function call: boolean JogMinus(TargetIP As String)

The result is true, if the execution was successful.

Description:
This function runs the jog mode in negative direction. For stopping this mode, call the function again! The jog mode behavior must be configured on the controller with LinMot Talk!

Method call: boolean setHomingBit(TargetIP As String, State as Boolean)

This method delivers true, if it passed successful.

Description:
This method set/reset bit 11 in Control Word using the value of “State”, which controls the homing procedure of the drive. The control of homing procedure must be done in user’s application! This bit must be reset when homing procedure has done, otherwise the drive is still in homing state!

Method call: boolean setSwitchOnBit(TargetIP As String, State as Boolean)

This method delivers true, if it passed successful.

Description:
This method set/reset bit 0 in Control Word using the value of “State”.

Method call: boolean setErrorAcknowledgeBit(TargetIP As String, State as Boolean)

This method delivers true, if it passed successful.

Description:
This method set/reset bit 7 in Control Word using the value of “State”.

Method call: boolean setBit0(TargetIP As String, State as Boolean)

This method delivers true, if it passed successful.

Description:
This method set/reset bit 0 in Control Word using the value of “State”.

▼
▼
▼

Method call: boolean setBit15(TargetIP As String, State as Boolean)

This method delivers true, if it passed successful.

Description:
This method set/reset bit 15 in Control Word using the value of “State”.

Method call: boolean setJogPlus(TargetIP As String, State as Boolean)

This method delivers true, if it passed successful.

Description:
This method set/reset bit Jog + in Control Word using the value of “State”.

Method call: boolean setJogMinus(TargetIP As String, State as Boolean)

This method delivers true, if it passed successful.

Description:
This method set/reset bit Jog - in Control Word using the value of “State”.

Method call: boolean MasterSlaveHoming(TargetIP As String)

This method delivers true, if it passed successful.

Description:
This function performs a homing procedure, comparable with the standard function „Homing()“. In addition, homing state of connected slaves will be handled as well.
If homing procedure need more than 5 minutes, a timeout error will be set in the DLL error (“Timeout During Homing”).

HINT: This function will work only, if Master/Slave application is installed on the drives and minimum one slave is connected to the master!

Method call: StateMAchineStates getStateMachineState(TargetIP As String)

This method delivers the actual state machine state.

Enumeration

```
Enum StateMachineStates As UShort
        NotReadyToSwitchOn = 0
        SwitchOnDisabled = 1
        ReadyToSwitchOn = 2
        SetupError = 3
        GeneralError = 4
        HWTests = 5
        ReadyToOperate = 6
        OperationEnabled = 8
        Homing = 9
        HomingFinished = 90
        ClearanceCheck = 10
        ClearanceCheckFinished = 100
        GoingToInitialPosition = 11
        GoingToInitialPositionFinished = 110
        Aborting = 12
        Freezing = 13
        QuickStopOnError = 14
        GoingToPosition = 15
        GoingToPositionFinished = 150
        JoggingPlus = 16
        Jogging_MovingPositive = 161
        JoggingPlusFinished = 162
        JoggingMinus = 17
        Jogging_MovingNegative = 171
        JoggingMinusFinished = 172
        Linearizing = 18
        PhaseSearch = 19
        SpecialMode = 20
        BrakeDelay = 21
End Enum
```

2.3.3 Motion commands, parameter access in detail

Note: All motion commands described in this chapter delivers a boolean information, true for successful handling of the command, false, if the function had a malfunction. The necessary count nibble is handled DLL internal, so there is no need to care about! To enter the motion command in the generic function, set a zero for the count nibble!

Function call: boolean LMmt_MoveAbs(TargetIP As String, Pos1 As Single, MaxVel1 As Single, Acc1 As Single, Dec1 As Single)

This function make an absolute movement (absolute to home position) to the specified position (Pos1). The MaxVel1 specifies the maximum velocity of the movement, Acc1 the maximum acceleration, Des1 the maximum deceleration. The TargetIP specifies the drive which is selected for the movement.

Function call: boolean LMmt_MoveRel(TargetIP As String, Pos1 As Single, MaxVel1 As Single, Acc1 As Single, Dec1 As Single)

This function make a relative movement (absolute to actual position) with the specified distance (Pos1). The MaxVel1 specifies the maximum velocity of the movement, Acc1 the maximum acceleration, Des1 the maximum deceleration. The TargetIP specifies the drive which is selected for the movement.

Function call: boolean LMmt_StartCTCommand(TargetIP As String, CTEntryID As UInteger)

This function starts the command table with the specified EntryID.

Function call: boolean LMmt_ClearEventEvaluation(TargetIP As String)

This function call will clear an active trigger (e.g. motion command which defines a trigger to execute). It is possible to cancel a running command table. Depending on the command table execution, it may be an error “Unknown Motion Command” can occur. A command table should be terminated by itself, using a command line with no new target ID.

Function call: boolean LMmt_Stop(TargetIP As String, Decceleration As Single)

This function stops the actual movement on the drive defined by TargetIP with the defined deceleration (m/s²).

Function call: boolean LMmt_WriteLivePar(TargetIP As String, UPID As UInteger, UPIDValue As Integer)

This function allow write access to the specified UPID on the drive specified by TargetIP. Only RAM values with runtime access can be set!

Function call: boolean LMav_Mod16BitCTPar(TargetIP As String, CTEntryID As Integer, ParaOffset As Integer, ParaValue As Integer)

This command allows to modify a 16 bit parameter in the command table (only RAM modification) on E series. Select the command table line with CTEntryID, ParaOffset defines an offset value (see list below) and ParaValue writes the value of the parameter.

Offset:
2 = Linked ID (the ID for the following line which should be used)
4 = Motion Command Header
6 = first motion command parameter

Note: For detailed information use the manual “Motion Control Software”

Function call: boolean LMav_Mod32BitCTPar(TargetIP As String, CTEntryID As Integer, ParaOffset As Integer, ParaValue As Integer)
This command allows to modify a 32 bit parameter in the command table (only RAM modification) on E series. Select the command table line with CTEntryID, ParaOffset defines an offset value (see list below) and ParaValue writes the value of the parameter.

Offset:
6 = first motion command parameter
10 = second motion command parameter

Note: For detailed information use the manual “Motion Control Software”

Function call: boolean LMav_RunCurve(TargetIP As String, CurveID As Integer, CurveOffset As Integer, TimeScale As Integer, AmplitudeScale As Integer)

This command allows to start a predefined curve which is stored on the controller. The CurveID specifies the ID of the stored curve, the CurveOffset defines the curve offset, TimeScale allows to scale the curve (0...200%) and the AmplitudeScale scales the amplitude from -2000% … 2000%. The value „TimeScale“ is scaled with 0.01%, the value „AmplitudeScale“ with a scale of 0.1%.

Function call: boolean LMav_MoveBestehorn(TargetIP As String, Position As Single, Velocity As Single, Acceleration As Single, Jerk As Single)

With this command you can make an absolute movement with a Bestehorn-profile for limited jerk (only supported on X12x0 and E14x0 series). The target position is set with Position (in mm), the maximum velocity with Velocity (in m/s), the acceleration with Acceleration (in m/s²) and the jerk with Jerk (in m/s³).

Function call: boolean LMav_MoveBestehornRelative(TargetIP As String, Position As Single, Velocity As Single, Acceleration As Single, Jerk As Single)

With this command you can make an relative movement/demand position increment with a Bestehorn-profile for limited jerk (only supported on X12x0 and E14x0 series). The target position is set with Position (in mm), the maximum velocity with Velocity (in m/s), the acceleration with Acceleration (in m/s²) and the jerk with Jerk (in m/s³).

Function call: boolean LMav_MoveSin(TargetIP As String, Position As Single, Velocity As Single, Acceleration As Single)

With this command you can make an absolute movement with a sinoid-profile (only supported on X12x0 and E14x0 series). The target position is set with Position (in mm), the maximum velocity with Velocity (in m/s), the acceleration with Acceleration (in m/s²).

Function call: boolean LMav_MoveSinRelative(TargetIP As String, Position As Single, Velocity As Single, Acceleration As Single)

With this command you can make an relative movement/demand position increment with a sinoid-profile (only supported on X12x0 and E14x0 series). The target position is set with Position (in mm), the maximum velocity with Velocity (in m/s), the acceleration with Acceleration (in m/s²).

Function call: boolean LMmt_GenericMC(TargetIP As String, Parameters according Table 2)

This command can be used to handle any motion command which is described in the manual „Motion Control Software“. You set the motion command header in the MCHeader and the necessary parameters in the MCParaWordX parameters. The MCHeader is a hexadecimal value described in the manual. You need to enter this as a decimal value or with a conversion function. The count nibble will be generated DLL internal! You must enter only the motion command header value, set x to 0.

MCHeader	MCParaWord0	MCParaWord1	MCParaWord2	MCParaWord3	MCParaWord4
UInt16	UInt16				
UInt16	UInt16	Uint16			
UInt16	UInt16	SInt16			
UInt16	UInt16	UInt32			
UInt16	UInt16	SInt32			
UInt16	SInt32	UInt32	UInt32	UInt32	
UInt16	UInt32				
UInt16	UInt32	UInt32	UInt32		
UInt16	SInt32				
UInt16	SInt32	SInt32			
UInt16	SInt32	SInt32	SInt32		
UInt16	UInt16	SInt32	SInt32		
UInt16	UInt16	SInt32	UInt16	SInt16	
UInt16	UInt16	SInt32	UInt32	SInt16	
UInt16	UInt16	UInt16	SInt16		
UInt16	UInt16	UInt16	SInt32		
UInt16	UInt16	UInt16	SInt32	SInt32	
UInt16	UInt16	UInt16	SInt32	SInt32	SInt32
UInt16	UInt16	SInt32	SInt32	SInt32	
UInt16	UInt16	SInt32	SInt16	SInt32	
UInt16	SInt16	UInt16	UInt16	UInt16	
UInt16	SInt16				
UInt16	SInt32	UInt32			
UInt16	SInt32	UInt32	UInt32		
UInt16	SInt32	UInt32	UInt32	UInt16	
UInt16	SInt32	UInt32	UInt32	UInt32	
UInt16	SInt16	UInt16	UInt16		
UInt16	UInt32	UInt32			
UInt16	UInt16	SInt32	SInt32	SInt32	SInt32
UInt16	SInt16	SInt32			
UInt16	SInt32	UInt16			
UInt16	SInt32	UInt16	UInt16		
UInt16	UInt16	UInt16	UInt16	UInt16	
UInt16	SInt32	UInt32	UInt32	SInt16	SInt16
UInt16	SInt32	UInt32	UInt32	UInt32	UInt32

Table 2: Parameter options for generic function use

Example:

For example motion command VAI Go To Pos (010xh). The „x“ is the placeholder for the count nibble. By using this DLL, the count nibble is handled internal. To enter a motion command, just set a „0“ for the „x“. In this example you will send: 0100h (256 decimal) as MCHeader.

Notice: The content of the parameter words must be coded as describes in the MC software manual!

Additional example:

4.3.55 Time Curve To Pos With Adjustable Time On Rising Trigger Event (04Exh)

Name	Byte Offset	Description	Type	Unit
Header	0	04Exh: Time Curve Go To Absolute Position With Adjustable Time On Rising Trigger Event	UInt16	-
1. Par	2	Curve ID	UInt16	1..100
2. Par	4	Target Position	SInt32	0.1 um
3 Par	8	Curve Time	SInt32	10us

	Data typ	Decimal	Hex
MCHeader	UInt16	1248	04E0
Curve ID = 1	UInt16	1	0001
Position = 50mm	SInt32	500000	7A120
Time = 750ms	SInt32	75000	124F8

Function call, Type Cast for example in Visual Basic:
.LMmtGenericMC(«192.168.0.1», CUint16(1248), CUInt16(1), CInt32(500000), CInt32(75000))

Notice: The raw data of all parameter words must be delivered in the specified data type and scale according to the command description in the “User Manual Motion Control SW”!

If variables are used, declare each in the specified data type!
If values are entered direct, use suitable type cast to prevent wrong casting of data!

4.3.114 VAI Dec=Acc Go To Pos With Max Curr (0C5xh)

Name	Byte Offset	Description	Type	Unit
Header	0	0C5xh: VAI Dec=Acc Go To Pos With Max Curr	UInt16	-
1. Par	2	Target Position	SInt32	0.1 um
2. Par	6	Maximal Velocity	UInt32	1E-6 m/s
3. Par	10	Acceleration = Deceleration	UInt32	1E-5 m/s^2
4. Par	14	Demand Curr Limit	Uint16	1mA

	Data typ	Decimal	Hex
MCHeader	UInt16	3152	0C50
Target Position = 50	SInt32	500000	7A120
Max Velocity = 2m/s	UInt32	2000000	1E8480
Acc = Dec = 3m/s3	UInt32	300000	493E0
Current Limit = 2A	UInt16	2000	7D0

.LMmtGenericMC(«192.168.0.1», CUint16(3152), CInt32(500000), CUInt32(2000000), CUInt32(300000), CUInt16(2000))

Function call: boolean LMfc_ChangeTargetForce(TargetIP As String, TargetForce As Single)

This command is used in force control mode. The command set the target force (in N). This function is only available with the technology function „Force Control“ order no.0150-2503.

Function call: boolean LMfc_GoToPosForceCtrlHighLim(TargetIP As String, Position As Single, Velocity As Single, Acceleration As Single, ForceLimit As Single, TargetForce As Single)

This command is used in force control mode. The command starts the movement to the specified position (in mm) with the defined maximum velocity and acceleration. If the defined force limit is reached, the drive switch to force control mode and use the force set point defined with Target Force (in N). This function is only available with the technology function „Force Control“ order no.0150-2503.

Function call: boolean LMfc_GoToPosForceCtrlLowLim(TargetIP As String, Position As Single, Velocity As Single, Acceleration As Single, ForceLimit As Single, TargetForce As Single)

This command is used in force control mode. The command starts the movement to the specified position (in mm) with the defined maximum velocity and acceleration. If the actual force falls below the force limit, the drive switch to force control mode and use the force set point defined with Target Force (in N). This function is only available with the technology function „Force Control“ order no.0150-2503.

Function call: boolean LMfc_GoToPosRstForceCtrl(TargetIP As String, Position As Single, Velocity As Single, Acceleration As Single, Deceleration As Single)

This command is used in force control mode. The command resets the force control mode to normal mode by setting the movement to the specified position (in mm) with the defined maximum velocity acceleration and deceleration. This function is only available with the technology function „Force Control“ order no.0150-2503.

Function call: boolean LMfc_GoToPosRstForceCtrlSetI(TargetIP As String, Position As Single, Velocity As Single, Acceleration As Single, Deceleration As Single)

This function is used for force control mode. The function resets the force control mode to the normal position control mode, setting the I portion to the current value of the last force value. To do this, the specified position "Position" (in mm) is approached with a maximum velocity "Velocity" and acceleration "Acceleration" and deceleration "Deceleration". This function is only available with the "Force Control" technology function enabled Order number 0150-2503.

Function call: boolean LMfc_IncrementActPosAndResetForceControlSetI(TargetIP As String, Position As Single, Velocity As Single, Acceleration As Single, Deceleration As Single)

This function is used for force control mode. The function resets the force control mode to the normal position control mode, setting the I portion to the current value of the last force value. To do this, the current position "Position" (in mm) is incremented with a maximum velocity "Velocity" and acceleration "Acceleration" and deceleration "Deceleration". This function is only available with the "Force Control" technology function enabled (order number 0150-2503).

Function call: boolean LMfc_IncrementActPosWithHigherForceCtrlLimitAndTargetForce(TargetIP As String, Position As Single, Velocity As Single, Acceleration As Single, ForceLimit As Single, TargetForce As Single)

This function is used for force control mode.
The function increments the current position. When the force limit is reached, the drive switches to force control and adjusts to the target force.
This function is only available with the "Force Control" technology function enabled Order number 0150-2503.

Function call: string LMcf_GetErrorTxt(TargetIP As String)

This function delivers the actual pending error code as string. The error codes are listed in the “Motion Control Software Manual“.

Function call: integer LMcf_GetErrorCode(TargetIP As String)

This function delivers the actual pending error code as integer. The error codes are listed in the “Motion Control Software Manual“.

Function call: string LMcf_GetWarningTxt(TargetIP As String)

This function delivers the actual pending warning message. The warning messages are listed in the “Motion Control Software Manual“.

Function call: integer LMcf_GetWarningCode(TargetIP As String)

This function delivers the actual pending warning code. The warning codes are listed in the “Motion Control Software Manual“.

Function call: long LMcf_StartStopDefault(TargetIP As String, Mode As Integer)

This function can be used to control the drives firmware state by setting the mode:

Mode	Description
0	Reboot oft he controller
1	Set all ROM values to factory default (OS Software)
2	Set all ROM values to factory default (MC Software)
3	Set all ROM values to factory default (Interface Software)
4	Set all ROM values to factory default (Application Software)
5	Stop of Motion Command and Application software (necessary for some modes)
6	Start of Motion Command and Application Software

Table 3: Function modes for StartStopDefault

The return value is „0“, if the execution was successful, 2 if the command is still in execution. If 9090 is delivered, an entry in the DLL Error will be set (Timeout during configuration channel access). See chapter “Return values during parameter access” for details.

Function call: long getROM_ByUPID(TargetIP As String, UPID As Uint32)

This method get the value of the specified UPID from the drive ROM. The UPID need to be entered as UINT32, the address values normally are displayed in LinMot Talk coded in hexadecimal form, which need to be converted into integer value.

Function call: long getRAM_ByUPID(TargetIP As String, UPID As Uint32)

This method get the value of the specified UPID from the drive RAM. The UPID need to be entered as UINT32, the address values normally are displayed in LinMot Talk coded in hexadecimal form, which need to be converted into integer value.

Function call: long getMinVal_ByUPID(TargetIP As String, UPID As Uint32)

This method get the defined minimal value of the specified UPID from the drive. The UPID need to be entered as UINT32, the address values normally are displayed in LinMot Talk coded in hexadecimal form, which need to be converted into integer value.

Function call: long getMaxVal_ByUPID(TargetIP As String, UPID As Uint32)

This method get the defined maximum value of the specified UPID from the drive. The UPID need to be entered as UINT32, the address values normally are displayed in LinMot Talk coded in hexadecimal form, which need to be converted into integer value.

Function call: long getDefault_ByUPID(TargetIP As String, UPID As Uint32)

This Method get the defined default value of the specified UPID from the drive. The UPID need to be entered as UINT32, the address values normally are displayed in LinMot Talk coded in hexadecimal form, which need to be converted into integer value.

Function call: long setRAM_ByUPID(TargetIP As String, UPID As Uint32, Value As Long)

This Method set the value of the specified UPID to the drive RAM. The UPID need to be entered as UINT32, the address values normally are displayed in LinMot Talk coded in hexadecimal form, which need to be converted into integer value. The value to be set is in long format (the UPID defined scale need to be used to create the long value).

The return value is „0“, if the execution was successful, 2 if the command is still in execution. If 9090 is delivered, an entry in the DLL Error will be set (Timeout during configuration channel access). See chapter “Return values during parameter access” for details.

Function call: long setROM_ByUPID(TargetIP As String, UPID As Uint32, Value As long)

This Method set the value of the specified UPID to the drive ROM. The UPID need to be entered as UINT32, the address values normally are displayed in LinMot Talk coded in hexadecimal form, which need to be converted into integer value. The value to be set is in long format (the UPID defined scale need to be used to create the long value). The change get active after the next reboot!

The return value is „0“, if the execution was successful, 2 if the command is still in execution. If 9090 is delivered, an entry in the DLL Error will be set (Timeout during configuration channel access). See chapter “Return values during parameter access” for details.

Attention: The ROM memory modules allow several 100,000 write accesses! Use the write access in the ROM only for configuration purposes, never in each process steps!

Function call: long setRAM_ROM_ByUPID(TargetIP As String, UPID As Uint32, Value As long)

This method set the value of the specified UPID to the drive ROM and RAM. The UPID need to be entered as UINT32, the address values normally are displayed in LinMot Talk coded in hexadecimal form, which need to be converted into integer value. The value to be set is in long format (the UPID defined scale need to be used to create the long value). The change get active after the next reboot!

The return value is „0“, if the execution was successful, 2 if the command is still in execution. If 9090 is delivered, an entry in the DLL Error will be set (Timeout during configuration channel access). See chapter “Return values during parameter access” for details.

Attention: The ROM memory modules allow several 100,000 write accesses! Use the write access in the ROM only for configuration purposes, never in each process steps!

Function call: boolean LMav_SetCurrentCommandMode(TargetIP As String, Current As Integer)

This method change from the position controlled mode into the current command mode. In this mode, a current of 0 creates no force, a current with positive sign create a force in positive motor direction, a current with negative sign create a force in negative motor direction. The current has to be entered in milliampere. If a change in current is needed, the same method call with the new current can be done to set the new current. The status „Special Motion Active“ get true and can be checked by calling „isSpecialMotionActive“. If this mode is active, no other motion commands will be executed. The created force is proportional to the motor force constant and can be used for pressing applications (the force accuracy is around 10% on the same target position, if high precision is needed, please use a closed loop force control).

Function call: boolean LMav_SetCurrentCommandMode(TargetIP As String, Current As Single)

This method change from current command mode back to position controlled mode. The status „Special Motion Active“ get false and can be checked with „isSpecialMotionActive“. After „SetCurrentCommandMode“ is finished, the servo controller is working in position control mode. All motion commands can be used again.

2.2.4 Return values during parameter access

Error codes/Return values during parameter access:

Code Dezimal	Code Hex	Description
0	0h	OK, command executed
2	2h	Command is running
4	4h	Data block is writing (Curve service)
5	5h	Busy
192	C0h	UPID wrong
193	C1h	Parameter data type wrong
194	C2h	Range error
195	C3h	Address usage wrong
196	C5h	Error: Command „Get next UPID List item“ was sent, without execution of „Start Getting UPID List“ before.
197	C6h	End of parameter list reached, no more entries available
208	D0h	Odd Address
209	D1h	Unit error (Curve service)
212	D4h	Curve always defined / Curve not available
9090	2382h	Timeout during parameter channel access

Table 4: Error codes and return values for parameter access

2.3.5 Curve access

Function call: boolean LMcf_LoadCurve(TargetIP As String, CurveID As Integer, SetpointCount As Integer, CurveName As String, Xcode As Byte, Ycode As Byte, Xlength As UInteger, Xdim As Integer, Ydim As Integer, Mode As Integer, Setpoints As Integer())

This command is used to load a curve into the drive curve memory. The CurveID specifies the curve number (1...100) which is necessary to get access to run the curve with LMav_RunCurve. The custom curve need to be defined as an array of Integers which is applied as set points. The SetpointCount is the count of set points which should be loaded (minimum 2). Every curve can be named with a 22 characters CurveName. Xcode and Ycode defines the dimensions of the curve:

Ycode		Xcode	
0	Position	0	Time
1	Velocity	1	Encoder Position [Inc]
2	Current	2	Position [mm]
3	Encoder Position		
4	Encoder Speed		
5	MicroSteps		

Table 5: Definition of the curve axis

Xlength defines the length of the X axis, the base length between 2 points is 10us or one increment. Xdim and Ydim defines the axis units.:

XDim & YDim	Scale unit	Description
5	0,1um	Standard linear Position Unit
26	0,01ms	Standard Curve Timebase
27	1 Inkrement	Standard Encoder Position Unit

Table 6: Definition of the curve axis intervals

The mode defines the operation mode of this function. The following modes are implemented:

Mode	Description
0	Load curve into RAM
1	Delete all curves in RAM and ROM
2	Store all curves from RAM to ROM

Table 7: Function modes for loading curves

For loading curves, first all curves need to be deleted. Call this function in mode 1, except of the TargetIP all other values can be set with some values, they are not used for this mode.
After the curves are deleted (function response changes from true back to false) the new curves can be loaded into the RAM using mode 0 (during load procedure the response of the function call is true and get false if finished).
If required to store the curves into the ROM, the function need to be called in Mode 2 except of the TargetIP all other values can be set with some values, they are not used for this mode.
Note that the firmware must be stopped when saving to the ROM!

Note: Run only one curve download, don't load more than one curve at same time!

Function call: boolean LMcf_LoadCurve(TargetIP As String, Mode As Integer, CurveData As CurveDataDefinition)

This function is identical to the previous curve function. Only the curve data is summarized in a data structure as follows:

```
Structure CurveDataDefinition
        CurveID As UInt16
        SetpointCount As UInt16
        CurveName As String
        Xcode As Byte
        Ycode As Byte
        XLength As UInt32
        XDim As UInt16
        YDim As UInt16
        Setpoints As Integer()
End Structure
```

Function call: boolean LMcf_isCurveLoading(TargetIP As String)

This function can be used to check whether the loading/writing process of a curve is still ongoing.

Function call: integer LMcf_getCurveProgress(TargetIP As String)

This function provides feedback on the percentage loading progress of the curve both when writing to the drive and when loading from the drive (0-100% of setpoint points).

Function call: boolean LMcf_StartUploadCurve(TargetIP As String, CurveID As UInt16)

This function activates the loading of the specified curve from the drive into the application software. The loading process runs as a background service, as it takes some time to load a curve. This can be checked with ***LMcf_isCurveLoading()*** and ***LMcf_getCurveProgress()***. The return value ***True*** signals the successful start of loading. The result can then be read with ***LMcf_getUploadedCurveData()***.

Function call: boolean LMcf_StartDownloadCurve(TargetIP As String)

This function activates the loading of the specified curve from the application software into the drive. The loading process runs as a background service, as it takes some time to load a curve. This can be checked with ***LMcf_isCurveLoading()*** and ***LMcf_getCurveProgress()***. The return value True signals the successful start of loading.

Function call: CurveDataDefinition LMcf_getUploadedCurveData(TargetIP As String)

This function gets the curve previously loaded with ***StartUploadCurve()*** into a data structure ***CurveDataDefinition*** for further processing by the application software.

Function call: LMcf_setDownloadCurveData(TargetIP As String, CurveData As CurveDataDefinition)

This function sets the curve data with the data structure CurveDataDefinition for download into the Drive. To do this use ***LMcf_StartDownloadCurve()***.

Function call: integer() LMcf_getAllCurveID(TargetIP As String)

This function returns the curve ID's of all available curves as an integer array.

Function call: boolean LMcf_isCurveOnDrive(TargetIP As String, CurveID As UInt16)

This function can be used to check whether a curve ID exists on the drive.
True, if the curve exists, False, if the curve does not exist.

Function call: Boolean LMcf_DeleteAllCurvesInRAM(TargetIP As String)

This function deletes all curves in RAM.

Function call: Boolean LMcf_SaveAllCurvesFromRAMToFLASH(TargetIP As String)

This function stores the current RAM content into the FLASH memory of the drive.

Attention: The FLASH memory chips allow some 100,000 write accesses! Use the spokes in the ROM only for configuration purposes, never in every process step!

2.3.6 Command Table access

For temporary changes it is sufficient to make changes to the Command Table in RAM. Saving only makes sense if the Command Table is to be used without prior loading after a drive reboot.
For individual parameters there are the commands:
LMav_Mod32BitCTPar() and LMav_Mod16BitCTPar()

Used data structure:

```
Structure CTEntry
     data As List(Of Byte)
End Structure
Structure CommandTableStructure
      Entries As List(Of CTEntry)
      CTPresenceList As List(Of Byte)
End Structure
```

The content of the data structure is composed according to the definition in the manual «Drive Configuration over Fieldbus SG5-SG7».

Function call: CommandTableStructure `LMcf`_getCommandTableContent(TargetIP As String)

This function loads the Command Table from the Drive into the CommandTableStructure data structure.

Function call: Boolean `LMcf`_setCommandTableContent(TargetIP As String, CT As CommandTableStructure)

This function writes the data from the CommandTableStructure data structure into the Command Table RAM of the Drive.

Function call: UInt16 `LMcf`_DeleteCommandTable_RAM(TargetIP As String)

This function deletes the Command Table in the RAM of the drive.

Function call: UInt16 `LMcf`_WriteCommandTableToFLASH(TargetIP As String)

This function writes the Command Table RAM area into the FLASH.

Attention: The FLASH memory chips allow some 100,000 write accesses! Use the spokes in the ROM only for configuration purposes, never in every process step!

2.3.7 Drive parameter access

Function call: List(Of UPID_List) `LMcf`_getUPIDList(TargetIP As String, StartUPID As UShort, StopUPID As UShort)

This function generates a list of all UPID's from ***StartUPID*** to ***StopUPID*** and returns the UPID usage as data value. The decoding of the data value results as shown:

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
				Nicht für HASH verwendet				Live Parameterr					Rom Write	Rom Read	RAM Write	RAM Read

Function call: List(Of UPID_List) `LMcf`_getModified_UPIDList(TargetIP As String, StartUPID As UShort, StopUPID As UShort)

This function returns all UPID's and raw data values whose value no longer corresponds to the factory settings (default). This can be used to store the drive parameters like a configuration.
Determining the UPID`S starts from ***StartUPID*** to ***StopUPID***.
Refer to the manual «Drive Configuration over Fieldbus SG5-SG7» for an overview of UPID groups for specific functional areas of the drive firmware.
The entire address range extends from 0 to 0xFFFF.

Function call: Boolean `LMcf_setUPIDList` (TargetIP As String, UPIDList As List(Of UPID_List))

This function writes the raw data values of a UPID list to the specified drive.
This can be used to write a parameter configuration. In order to achieve defined states, the corresponding parameter range must be reset to factory settings (Default). This can be done with the command ***LMcf_StartStopReset()***.

2.3.8 Load / save drive configuration

Function call: Boolean Save_CommandTable(TargetIP As String, FilePath As String)

This function saves the Command Table of the selected Drive directly into a file. The data format is based on XML
The path can be freely defined as a file and its extension.

The return value is ***True*** if the file was saved successfully.

Function call: Boolean Save_CommandTable(CT As CommandTableStructure, FilePath As String)

This function saves a loaded Command Table data structure directly into a file. The data format is based on XML
The path can be freely defined as a file and its extension.

The return value is ***True*** if the file was saved successfully.

Function call: Boolean Load_CommandTable(TargetIP As String, FilePath As String)

This function loads a Command Table directly from the specified file created with ***Save_CommandTable()*** to the drive. The Command Table is only available in RAM! If you want it to be available after a restart, you must still save it with ***LMcf_WriteCommandTableToFLASH()***.

The return value on successful execution is ***True***.

Function call: CommandTableStructure Load_CommandTable(FilePath As String)

This function loads a stored Command Table into the Command Table data structure.

The return value on successful execution is ***True***.

Function call: Boolean Save_DriveParameters(TargetIP As String, FilePath As String)

This function saves all parameters from the specified drive that differ from the factory values (default) to the target file. The data format is based on XML
The path can be freely defined as a file and its extension.

The return value is ***True*** if the file was saved successfully.

Function call: Boolean Save_DriveParameters(Data As List(Of UPID_List), FilePath As String)

This function saves a UPID list to the destination file. The data format is based on XML
The path can be freely defined as a file and its extension.

The return value is ***True*** if the file was saved successfully.

Function call: Boolean Load_DriveParameters(TargetIP As String, FilePath As String)

This function loads parameters from the specified file created with ***Save_DriveParameters()*** to the addressed drive.

The return value on successful execution is ***True***.

Attention: This function sets all drive parameters to factory settings and then loads the parameters to the drive. Afterwards a restart must be triggered!

Function call: List(Of UPID_List) Load_DriveParameters(FilePAth As String)

This function loads parameters from the specified file created with ***Save_DriveParameters()*** into an UPID list.

The return value of successful execution is ***True***.

Function call: Boolean Save_Curves(TargetIP As String, FilePath As String)

This function saves all curves stored on the drive to a file. The data format is based on XML
The path can be freely defined as a file and its extension.

The return value is ***True*** if the file was saved successfully.

Function call: Boolean Load_Curves(TargetIP As String, FilePath As String)

This function loads the curves saved with ***Save_Curves()*** from the specified file directly into the Drive

The return value of successful execution is ***True***.

Function call: *Boolean* Save_DriveConfiguration(TargetIP *As String*, FilePath *As String*)

This function saves the entire drive configuration (including curves and Command Table) to the specified file.
The data format is based on XML
The path can be freely defined as a file and its extension.
This process can take a few minutes. The function does not take place in a separate process, but works blocking.

The return value is ***True*** if the file was saved successfully.

Function call: *Boolean* Load_DriveConfiguration(TargetIP *As String*, FilePath *As String*)

This function loads the specified file saved with ***Save_DriveConfiguration()*** to the drive.
Before that, the entire drive is reset to factory defaults.
This process can take several minutes. The function is not performed in a separate process, but works blocking.
To use the loaded configuration, the drive must be restarted! This can be done with ***LMcf_StartStopReset()***.

The return value of successful execution is ***True***.

Attention: This function resets the drive completely to factory settings and then loads the configuration data!

Function call: *UInt32* getDriveHash(TargetIP *As String*)

This function retrieves the hash value of the drive, which is calculated depending on the corresponding parameters of the drive during a reboot. The function ***LMcf_getUPIDList(TargetIP()*** can be used to load a list of all parameters. Bit 12 of the UPID data indicates whether the corresponding UPID is used in the HASH calculation or not.

Function call: *Boolean* isDriveConfigurationSame(TargetIP *As String*, FilePath *As String*)

This function uses a configuration previously saved with Save_DriveConfiguration() to check whether the specified drive still uses this configuration or whether the parameters have been changed.

Note: Already changes of controller parameters change the HASH!

Attention: The calculation of the HASH is only done during the start/ or reboot process.

Example scenario for automated configuration:

After commissioning or adjusting parameters, the application must trigger a restart of the drive via the DLL (***LMcf_StartStopReset()***). This recalculates the hash on the drive. Then the new configuration with the corresponding hash must be loaded and saved from the DLL (***Save_DriveConfiguration()***).

During normal operation, the ***isDriveConfigurationSame()*** function can then be used to check whether changes have been made to the drive configuration when the application software is restarted. If it is necessary for the application to use a consistent and unique configuration, the configuration can be reloaded to the drive with ***Load_DriveConfiguration()*** during the startup process. To activate this, the drive must also be restarted (***LMcf_StartStopReset()***).

Appendix I Commissioning LinUDP interface

For use of LinUDP the interface need to be configured.
After firmware installation, the interface is on factory defaults.
In this case:
- „Dis-/Enable“ is set to „Enable“ (needed to operate)
- „Ethernet Configuration “ → „IP Configuration Mode “ is set to S1 and S2 switches

You can select DHCP for automatic addressing with DHCP servers, or you can select „Static by IP Configuration“. The static IP can be entered in „IP Configuration“.

All monitoring channels are without any UPID. If needed, up to 4 channels can be configured to UPID addresses. The addressed values are transmitted in the answers and will be available by DLL functions.

Additional: „Master Configuration“ (change only if needed in special case)
In this part normally nothing need to be configured. Only if you want to change the communication ports or if you work with multiple computers running LinUDP in the same network.
Here you can change „Master Port“ and „Drive Port“ (slave).

Attention: Take care to use the modified port numbers by establishing the DLL connection!

If more than one master computer with LinUDP is used in the same network, you can configure IP filters.
You can choose „Single Master“, where the drive will filter all IP addresses only the first recognized IP during LinUDP communication is set as master system. Or you can set „Single Master with fix IP“ where you can define the master`s IP address in „Master IP Address“. All other master datagrams are ignored.

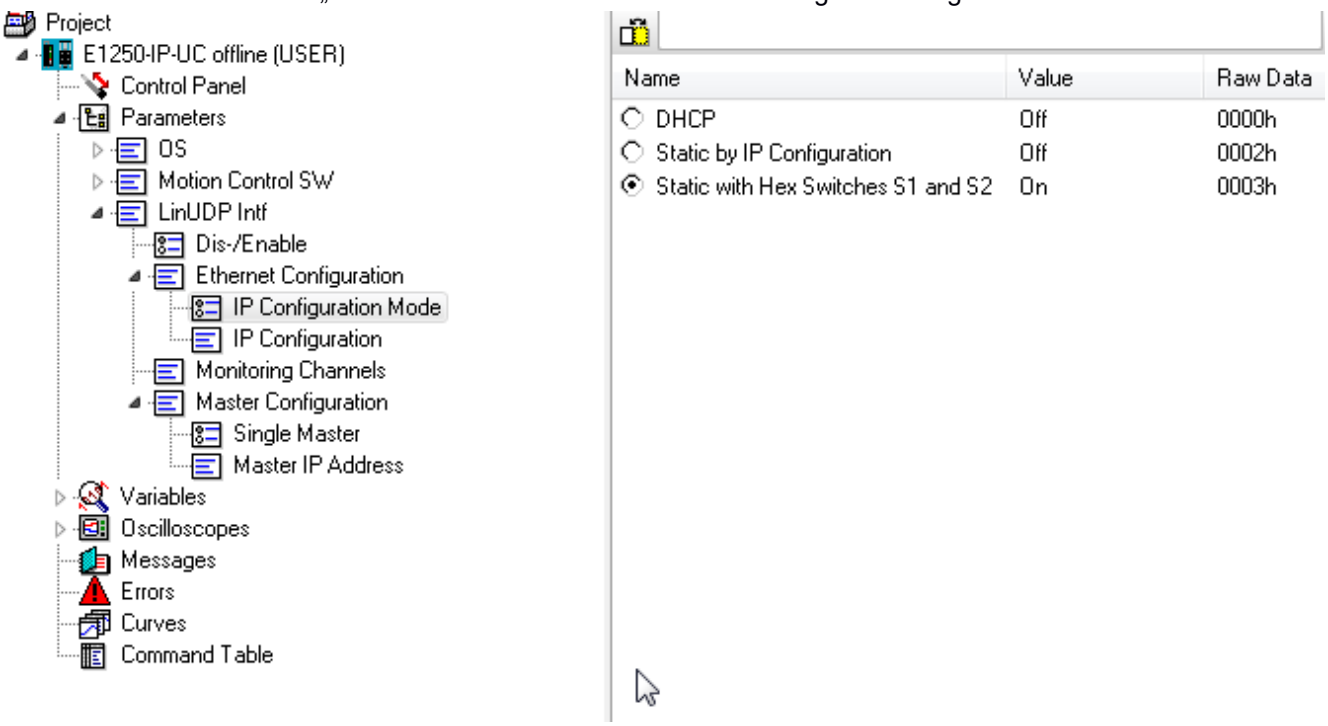
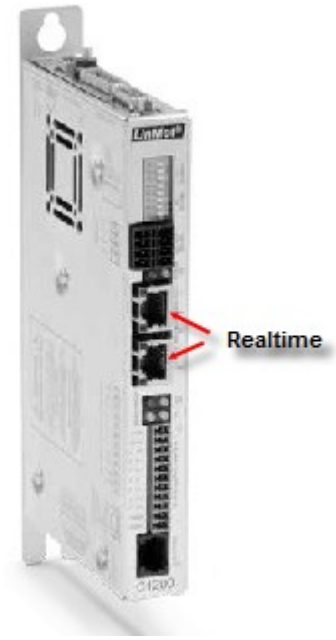


Figure 3: LinUDP Interface Configuration

Appendix II Realtime connection

Using LinUDP requires the necessary fieldbus connection. LinUDP is available on -LU servo controllers, which have different connection locations for the real time bus. The network can be used with standard network switches. The servo drives provide internally a two port switch for real time interface. Details are printed in the Installation Guide.



C1250-LU-XC-xS

E1250-LU-XC-xS

E1450-LU-XC-xS

Table 8: Realtime Connection Points

Daisy Chain Topology

Star Topology

Table 9: Network topology

Appendix III. LinUDP Testtool

The LinUDP test tool is a small software, which is installed during the DLL installation. With this test tool you can easily check the network connection to a drive and the interface configuration. The controller can be activated and homed. A simple motion to absolute position can be executed. If you uninstall this test tool, the DLL will be uninstalled as well!

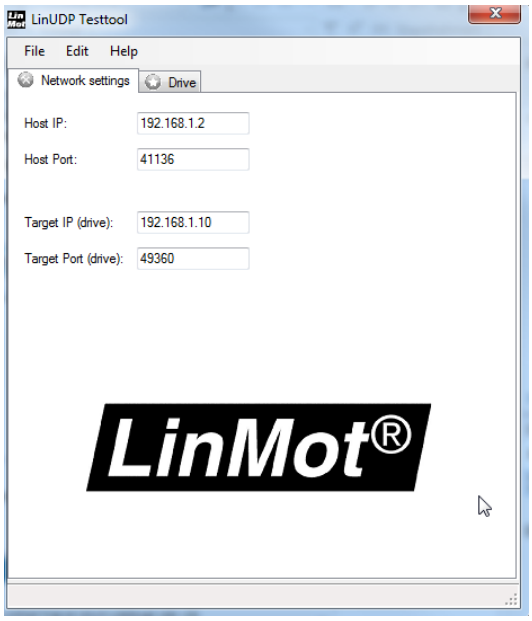


Table 10: Testtool – Network Settings

Function tab „Network settings“
Definition of computer IP address
Port is predefined to the predefined value on the controller

Definition of Target IP: Address of the controller
Port is predefined to the predefined value on the controller

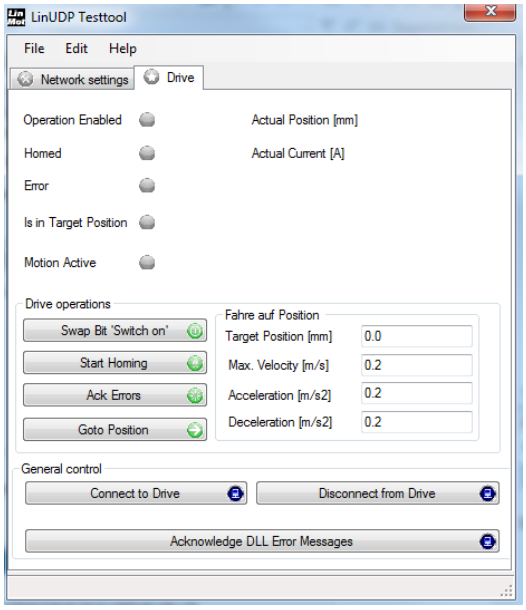


Table 11: Testtool - Drive

Function tab „Drive“
Overview of controller status bits
Actual position and actual current

Drive Operations:
Switching of bit 0 „SwitchOn“ to change the state machine on the controller.
Perform homing and acknowledge errors.
Move to absolute position.

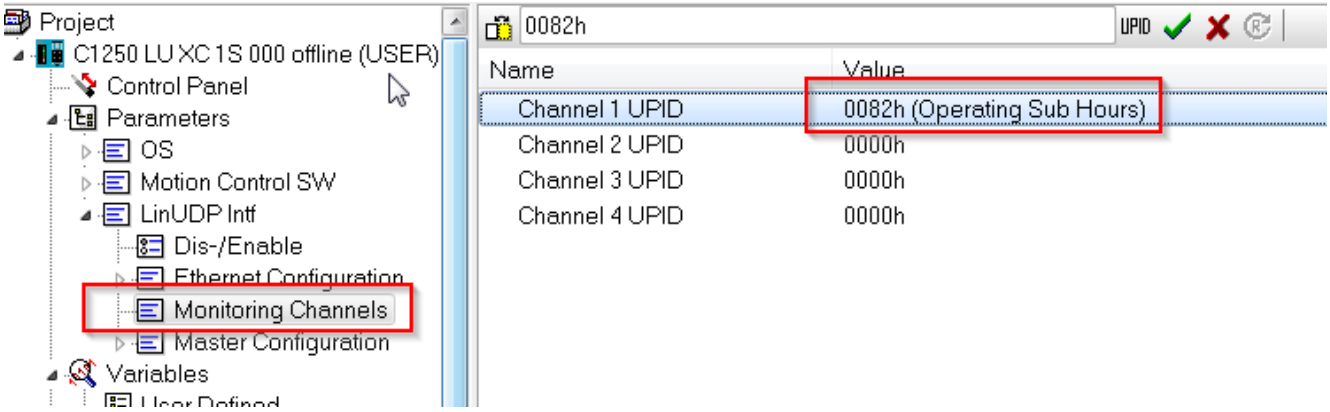
General control:
Establish connection and close connection

Acknowledge DLL specific error messages
DLL errors are displayed in the status bar.

In the menu section „Edit“ you can change the software language.
In the menu section „Help“ you can get the library description, get the version number of the DLL, check for new DLL versions and get the DLL installation path.

Appendix IV. Time stamp integration

LinUDP used on Windows PC is not deterministic. Based on that, recording measurements over time is not recommended. For some applications, it may be still suitable, to get a time stamp by using drive internal clock. This clock use a resolution of 1ms and can be easily added into a monitoring channel by its UPID 82h:



After this configuration, you will get for each response datagram the drive internal time stamp.

For use of the additional functions:
getActualPosWithTimestamp
getDemandPosWithTimestamp
getDemandCurrentWithTimestamp
getMonitoringChannelWithTimestamp

and the function of
«PseudoScope»

“Monitoring Channels” -> “Channel 1 UPID” must be set to **82**h (Operating Sub Hours)!

Appendix V. Manual Installation of LinUDP DLL driver

The DLL file installation is done manually by the user. For the installation you need admin rights!

Precondition for using the LinUDP DLL: You need to install Microsoft .NET 4.0 framework or newer, if you don`t have .NET installed! To check your installation with .NET and installing the DLL follow one of these sequences:

If you use Windows 7 then you need to modify two entries in the registry. Please follow chapter VI.I.

If you use Windows XP or older versions, please follow chapter VI.II.

This registration is necessary, if you want to use Microsoft Visual Studio or Excel for example. For use with LabVIEW, this installation is not required!

VI.I DLL Installation for Windows 7

Figure 4: Control Panel Windows

a. .Open your control panel:

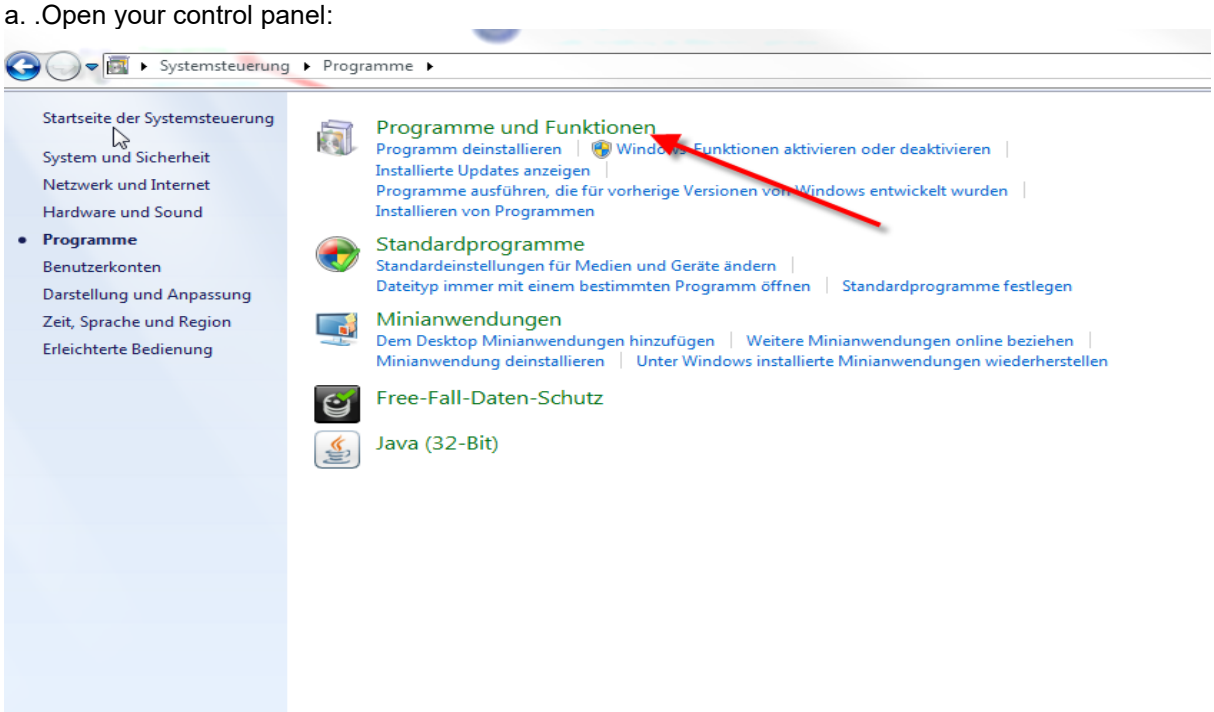


Figure 5: Control Panel Windows 7 Programs and Features

b. Click on programs
c. check for .NET Framework 4.0 (or higher).

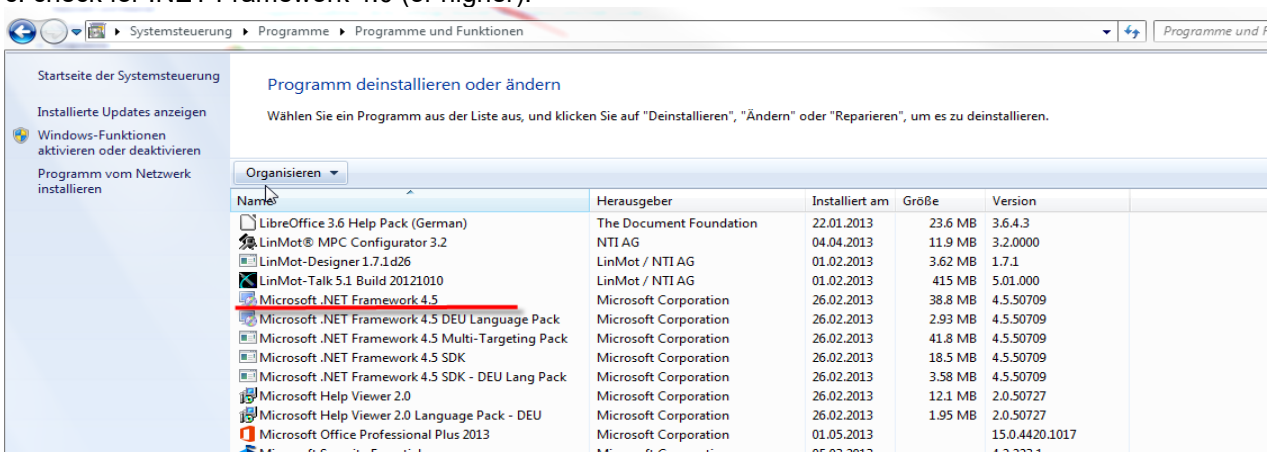


Figure 6: View Programs and Features Windows 7

If you have no .NET Framework 4.0 or higher, you need to install it with a Microsoft download.
You can use Google searching for “.net framework 4” or follow this link:

http://www.microsoft.com/en-us/download/details.aspx?id=17718

If .NET 4.0 or higher is available, you can copy the LinUDP.dll file in your windows\system32\ folder

In the next step you need to prepare your registry to get the dll proper registered.

You run Regedit to access your registry.

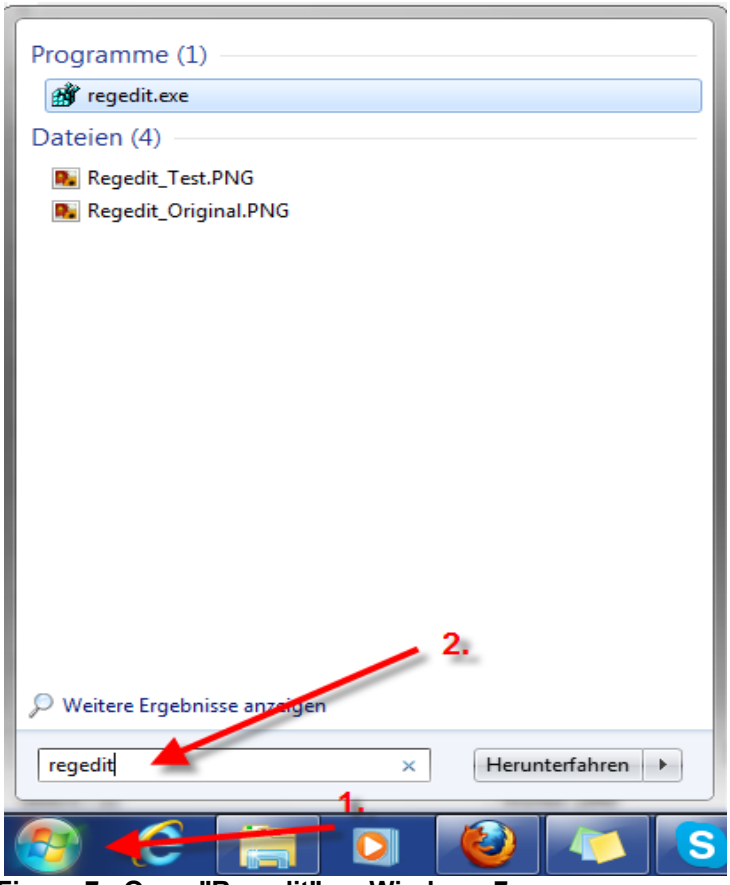


Figure 7: Open "Regedit" on Windows 7

Click on program and enter regedit, then press return. The following window will appear:

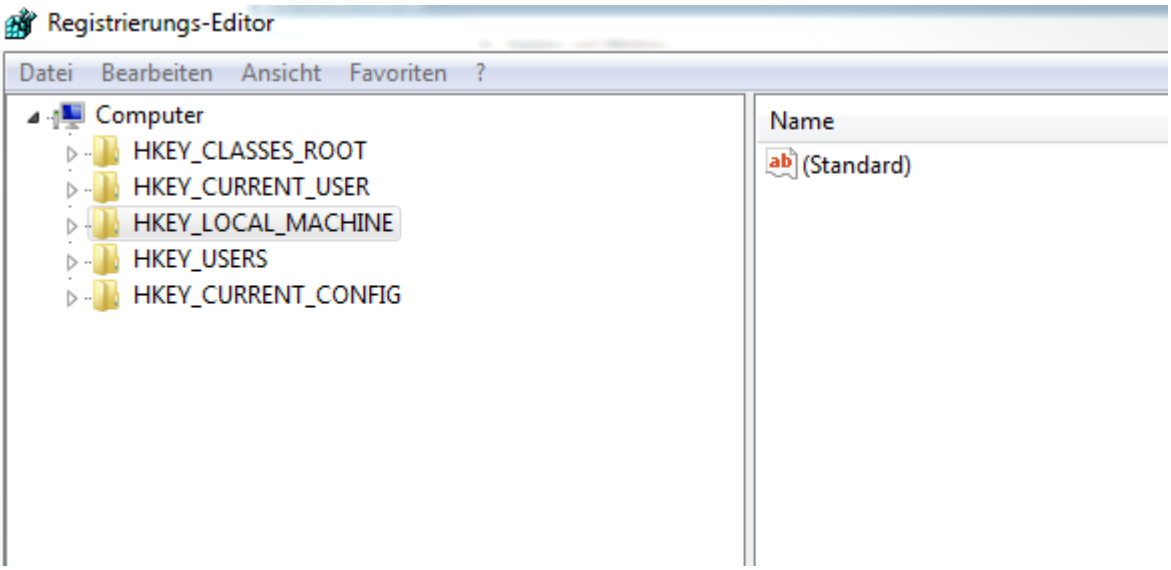


Figure 8: RegEdit Main View in Windows 7

Then open the following path:

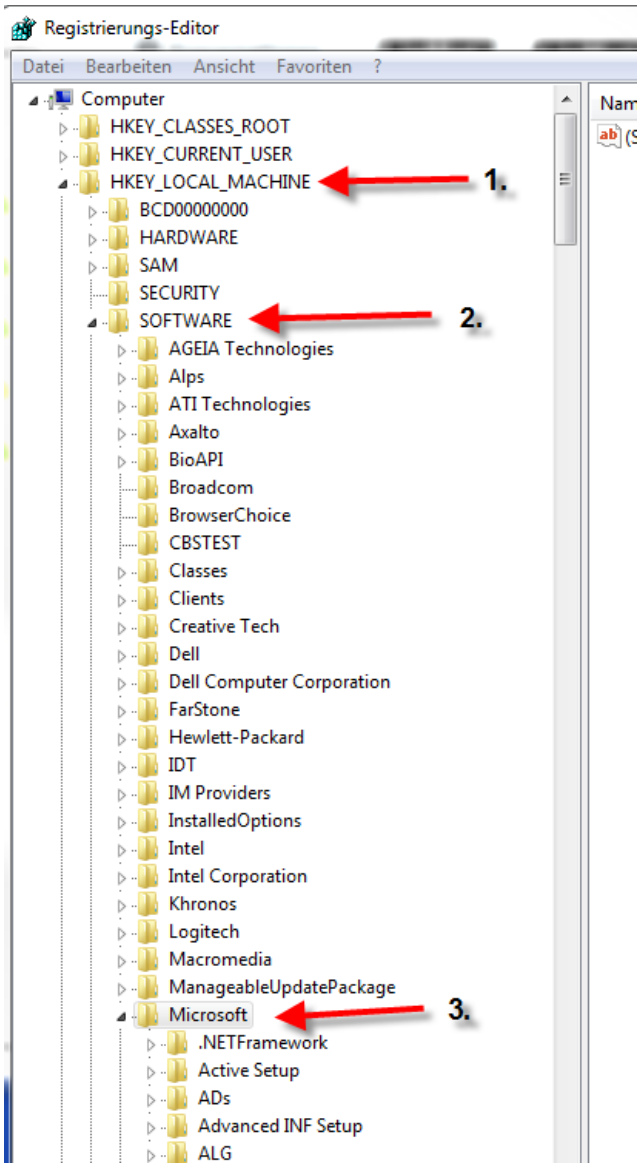


Figure 9: Registry Path Step 1

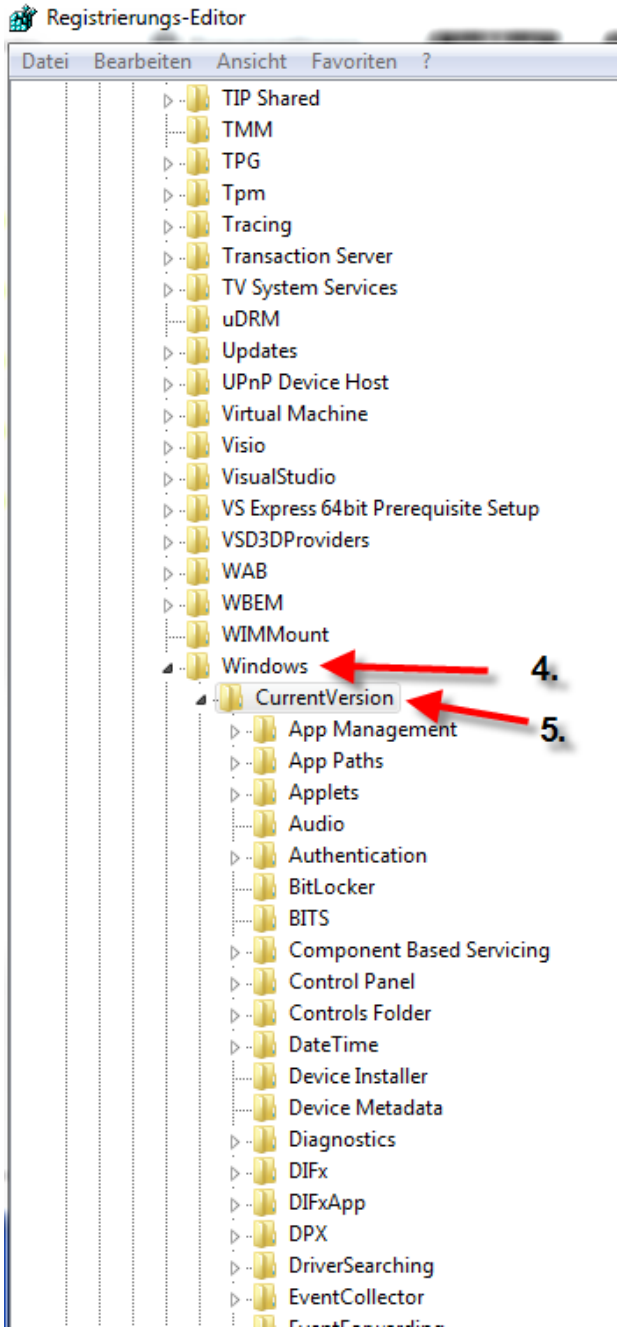


Figure 10: Registry Path Step 2

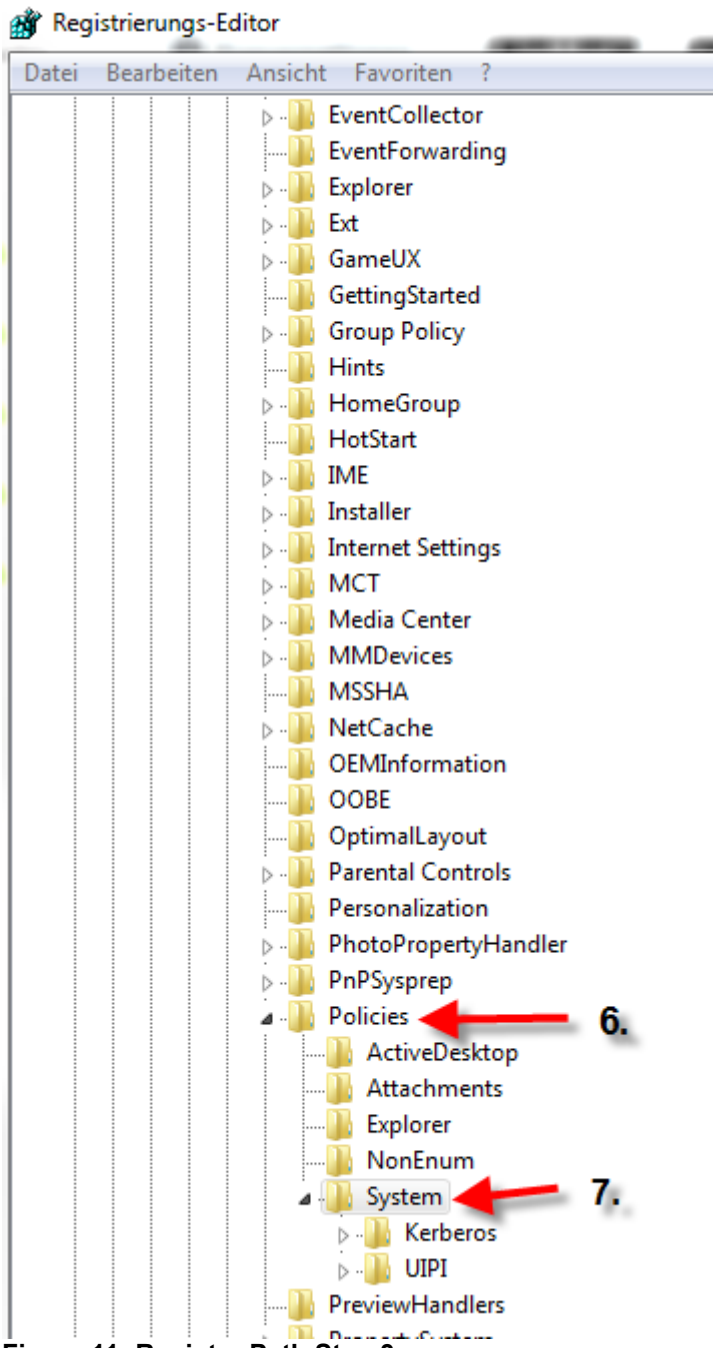


Figure 11: Registry Path Step 3

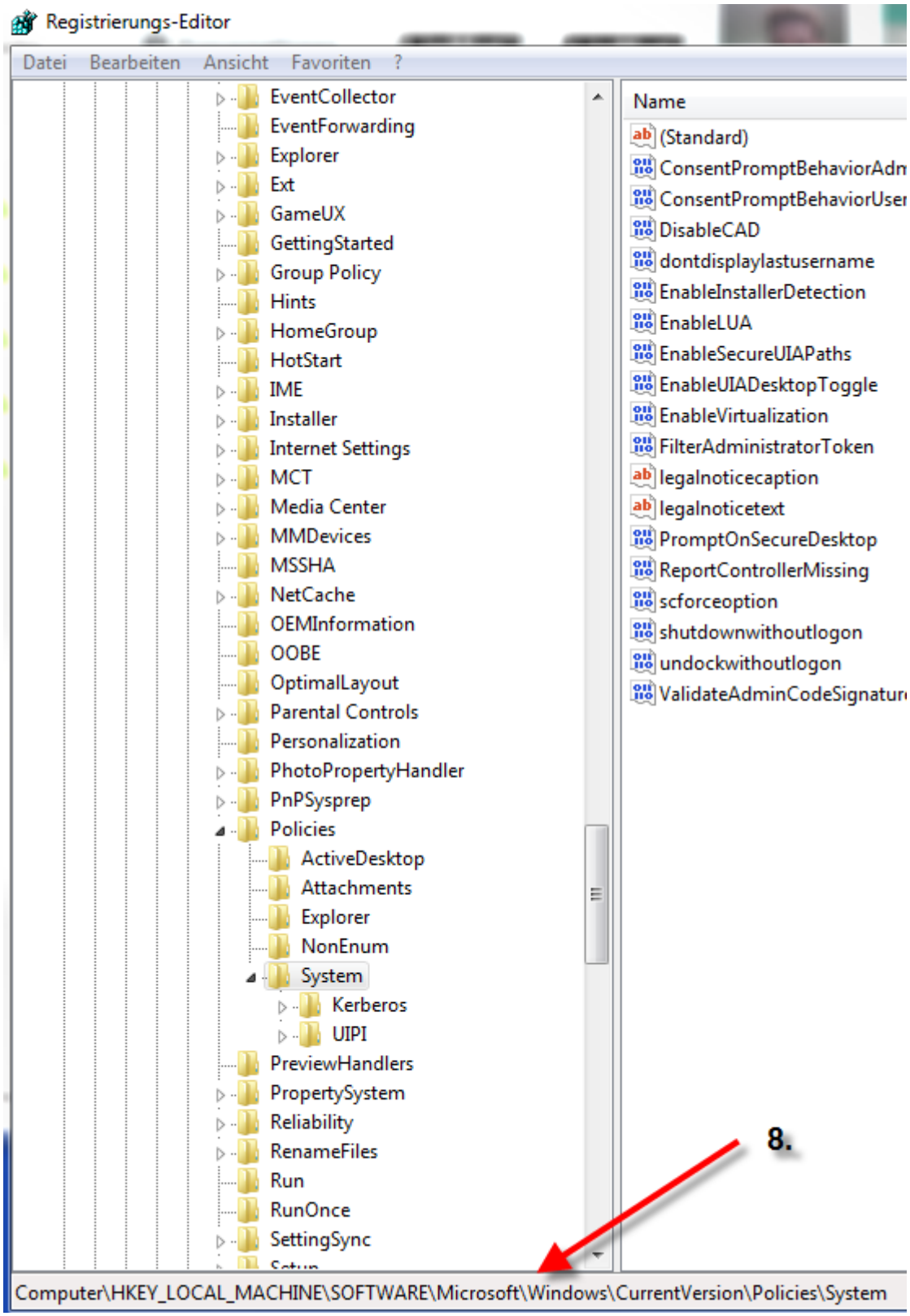


Figure 12: Registry Path Step 4

You should find the full path as 8.

Then modify the red marked entries:

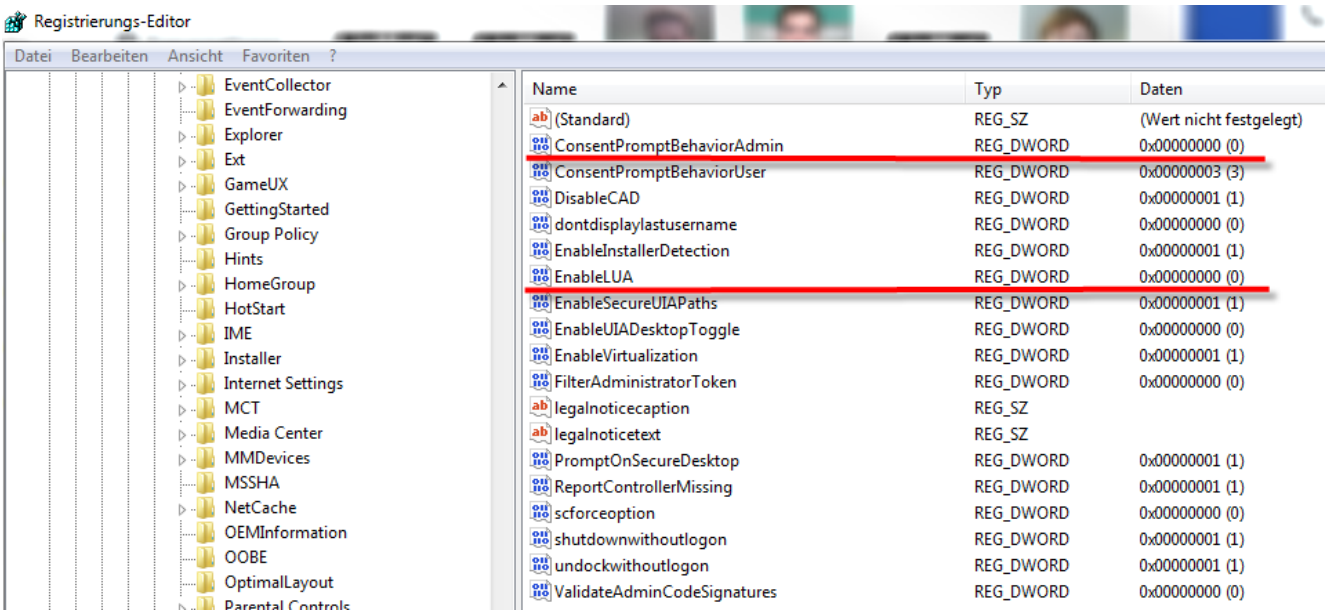


Figure 13: Modifications in the Windows 7 registry

The entries “ConsentPromptBehaviorAdmin” has to be “0” and “EnableLUA” has to be “0”.
You can first note down the original values of this entries, then you set these to “0” and reboot your computer.

Now you can register the dll into your systems configuration. To do so, open a command window and change the directory to the .NET folder (normally in Windows\Microsoft.NET\Framework\V4.xxxxx) like 1 and the v4.xxx like 2. If you are using Windows 7 64 Bit, please use the path Windows\Microsoft.net\Framework64\V4.xxxxx!

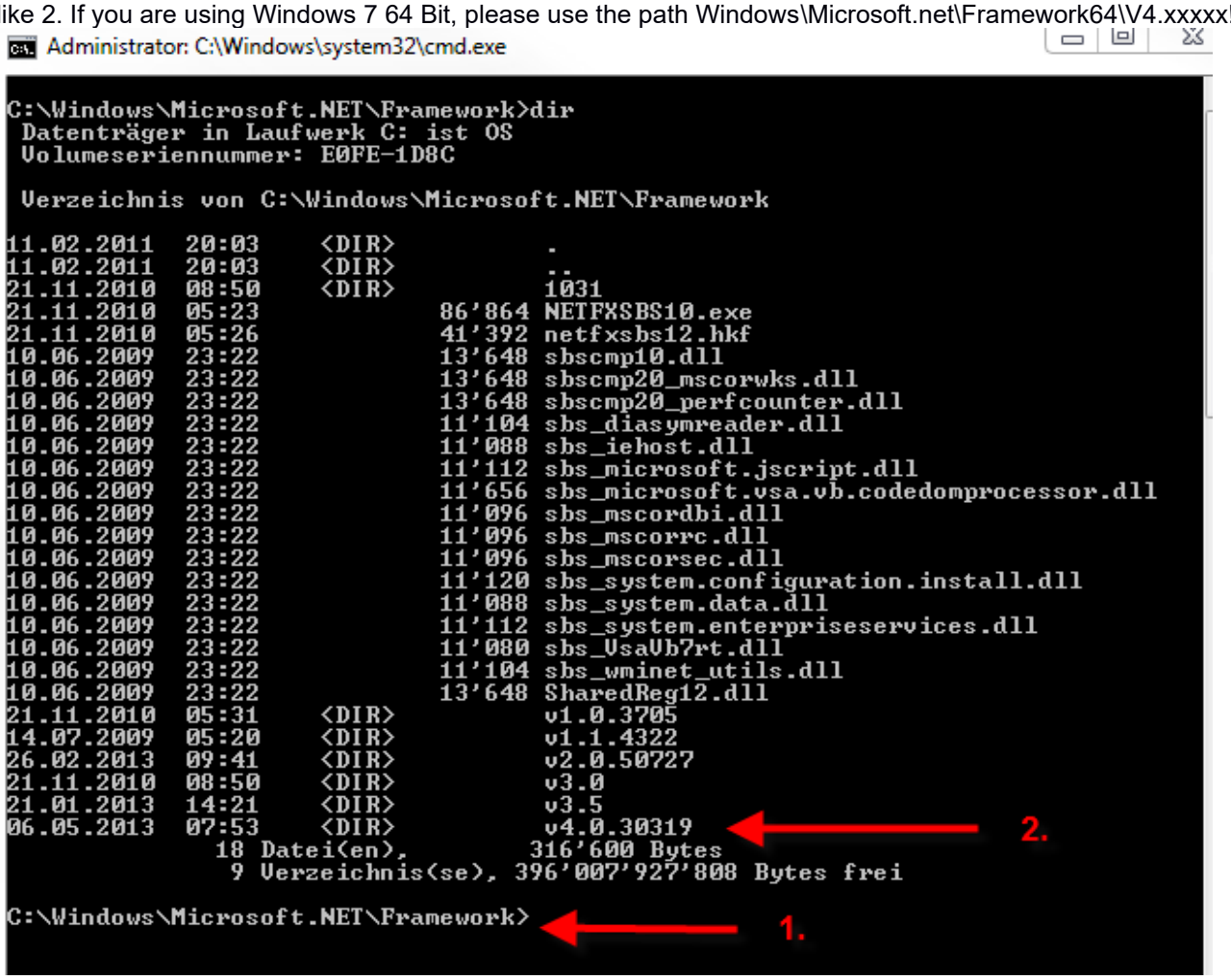


Figure 14: Registration of LinUDP.DLL Step 1

type now **“regasm c:\windows\system32\LinUDP.dll /tlb:LinUDP.tlb /codebase”**

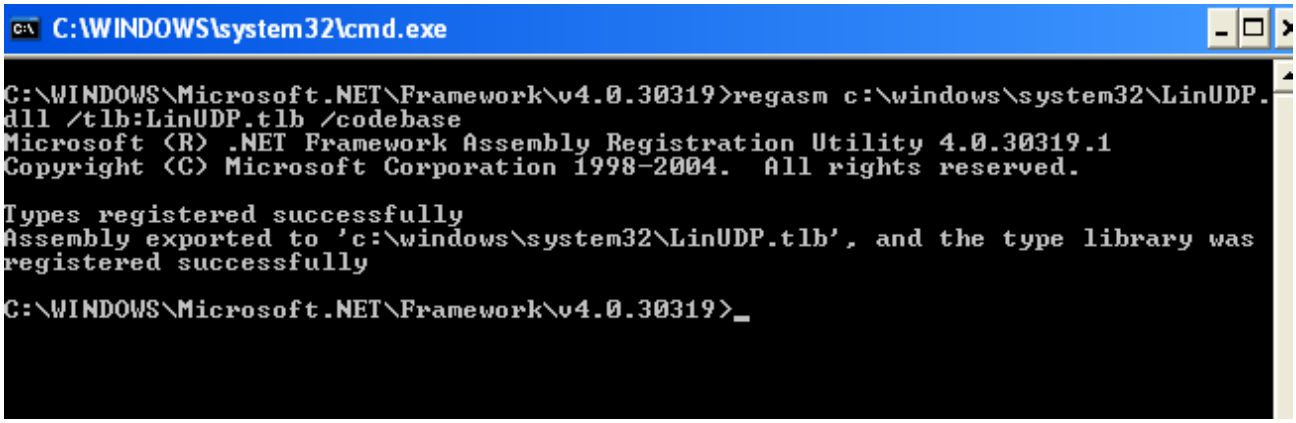


Figure 15: Registration of LinUDP.DLL Step 2

The registration should be done automatically.

Then you can restore the registry settings for ConsentPromptBehaviorAdmin and EnableLUA to the original values you have noted down. To activate the changes reboot your computer!

To use the Library in Microsoft Excel 2013 you start your Excel and activate the developer tab (1.)
Then you can click on “Visual Basic” (2.).

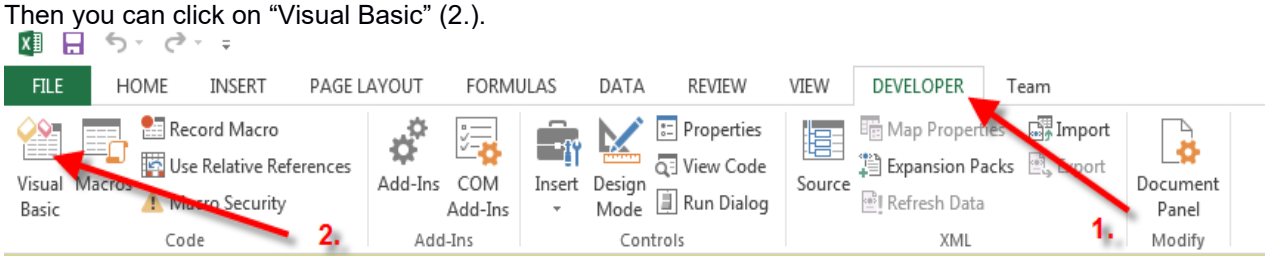


Figure 16: Excel 2013: Open Visual Basic Editor

The Visual Basic editor appears.
Here click on Tools (3.) and then on References (4.).

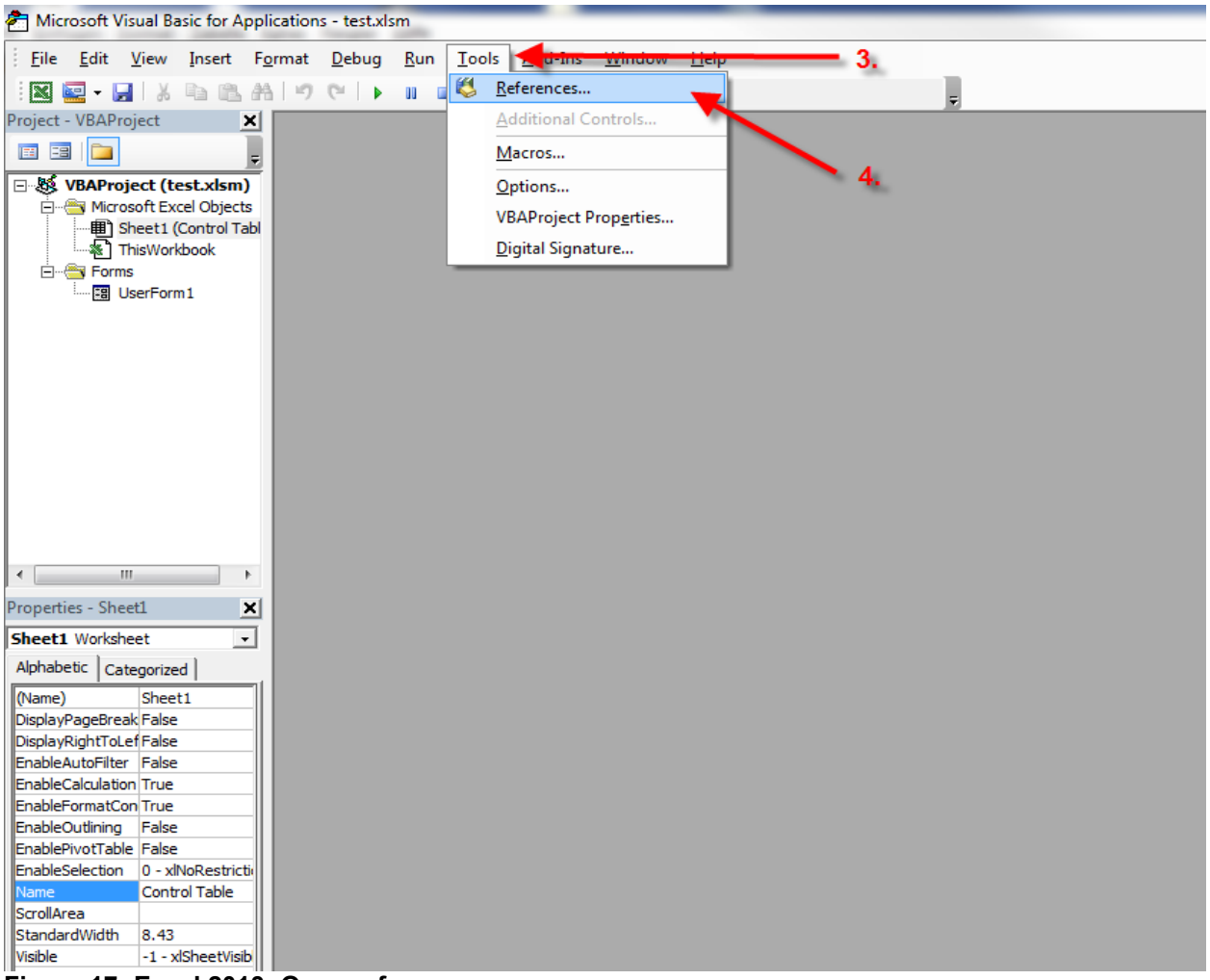


Figure 17: Excel 2013: Open reference group

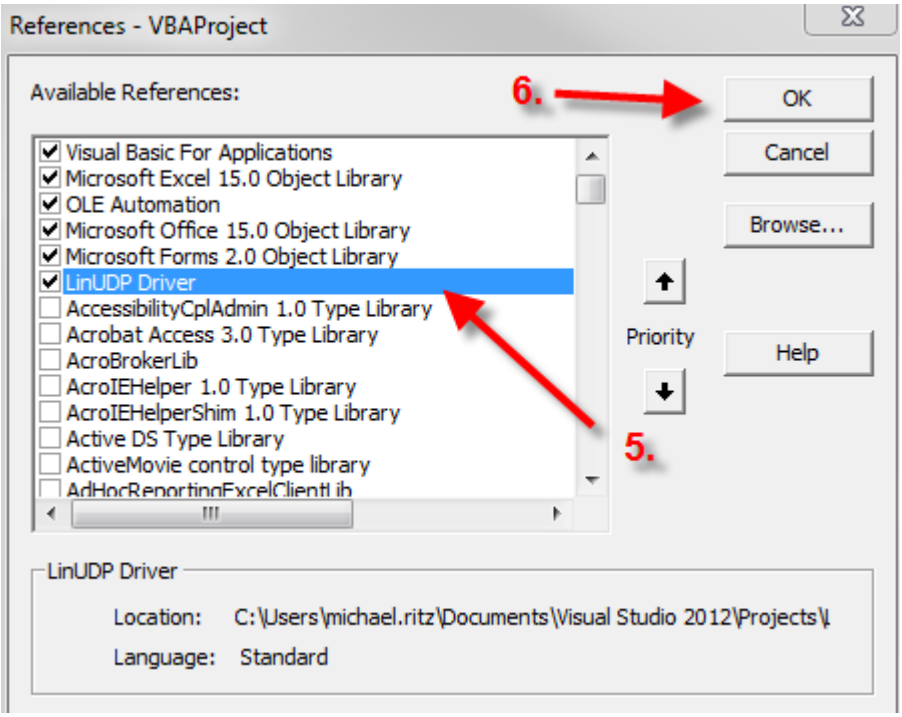


Figure 18: Excel 2013: Add reference for LinUDP

In the References window select the LinUDP Driver from the list (5.) and click OK (6.).
Now you can use LinUDP in this actual Excel Workbook! If you create a new Excel file, you have to select the

LinUDP driver again! This setting is only for the actual document and will be stored in the file. If you want to use this file on a second computer, LinUDP has to be installed separately for this second computer!

V.II. DLL installation for Windows XP or older

a. Open the control panel and click on Add or Remove Programs

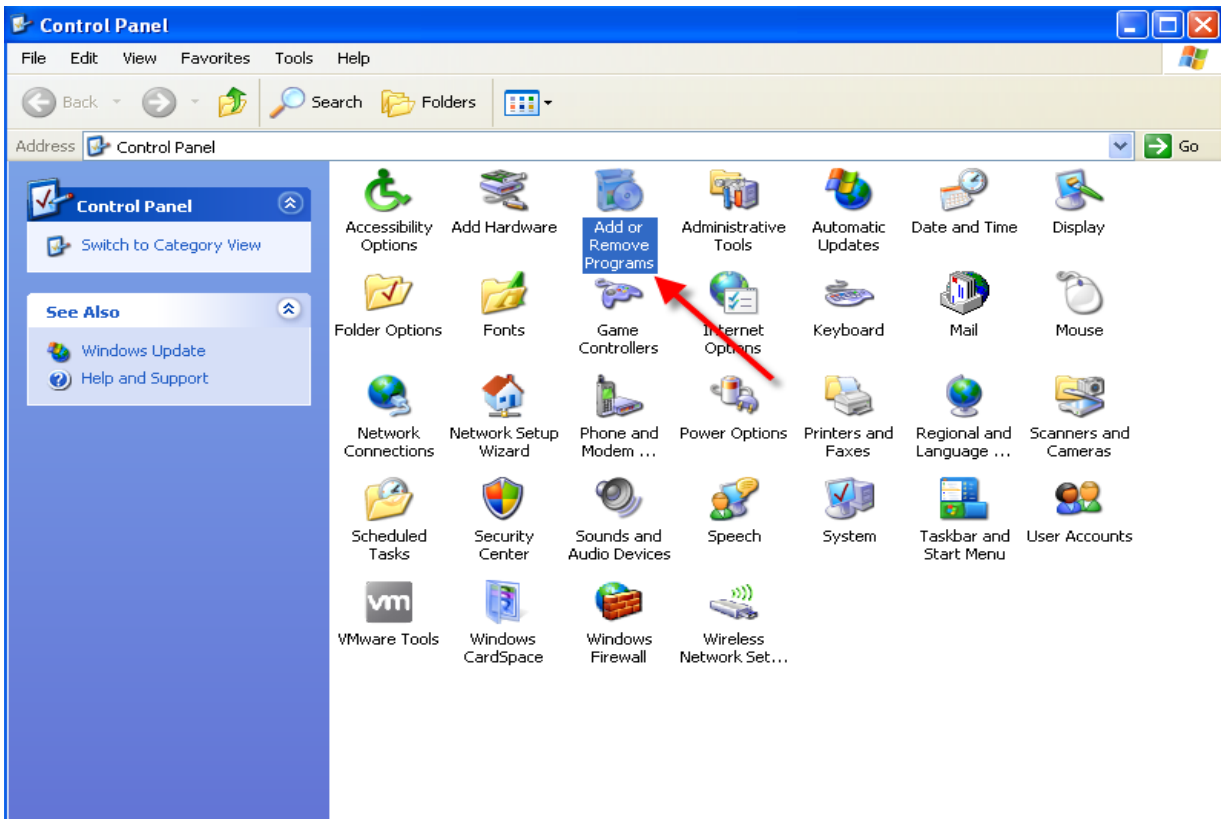


Figure 19: Control Panel Windows XP - Programs

b. Check for .NET framework 4.0 or higher

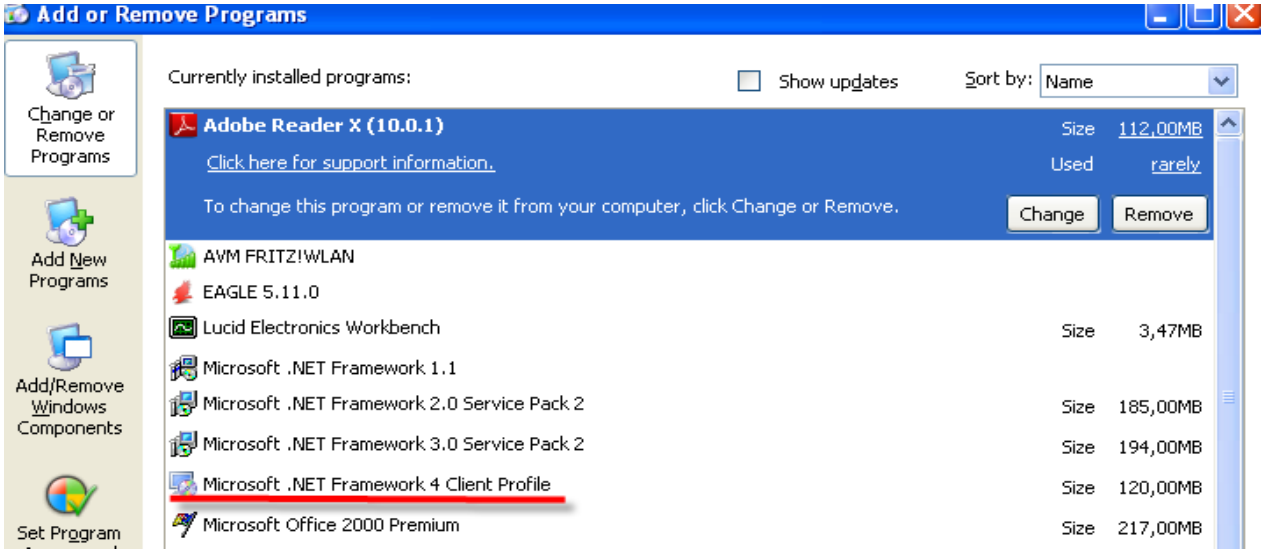


Figure 20: Control Panel Windows XP - Check for .NET Framework

If you have no .NET Framework 4.0 or higher, you need to install it with a Microsoft download.
You can use Google searching for “.net framework 4” or follow this link:

http://www.microsoft.com/en-us/download/details.aspx?id=17718

If .NET 4.0 or higher is available, you can copy the LinUDP.dll file in your windows\system32\ folder. Then you

need to register the dll in the windows system. To do this, please open a command window and change the path to the .NET framework folder like (1.) and (2.).

```
Administrator: C:\Windows\system32\cmd.exe

C:\Windows\Microsoft.NET\Framework>dir
 Datenträger in Laufwerk C: ist OS
 Volumeseriennummer: E0FE-1D8C

 Verzeichnis von C:\Windows\Microsoft.NET\Framework

11.02.2011  20:03    <DIR>          .
11.02.2011  20:03    <DIR>          ..
21.11.2010  08:50    <DIR>          1031
21.11.2010  05:23            86'864 NETFXSBS10.exe
21.11.2010  05:26            41'392 netfxsbs12.hkf
10.06.2009  23:22            13'648 sbscmp10.dll
10.06.2009  23:22            13'648 sbscmp20_mscorwks.dll
10.06.2009  23:22            13'648 sbscmp20_perfcounter.dll
10.06.2009  23:22            11'104 sbs_diasymreader.dll
10.06.2009  23:22            11'088 sbs_iehost.dll
10.06.2009  23:22            11'112 sbs_microsoft.jscript.dll
10.06.2009  23:22            11'656 sbs_microsoft.vsa.vb.codedomprocessor.dll
10.06.2009  23:22            11'096 sbs_mscordbi.dll
10.06.2009  23:22            11'096 sbs_mscorrc.dll
10.06.2009  23:22            11'096 sbs_mscorsec.dll
10.06.2009  23:22            11'120 sbs_system.configuration.install.dll
10.06.2009  23:22            11'088 sbs_system.data.dll
10.06.2009  23:22            11'112 sbs_system.enterpriseservices.dll
10.06.2009  23:22            11'080 sbs_VsaVb7rt.dll
10.06.2009  23:22            11'104 sbs_wminet_utils.dll
10.06.2009  23:22            13'648 SharedReg12.dll
21.11.2010  05:31    <DIR>          v1.0.3705
14.07.2009  05:20    <DIR>          v1.1.4322
26.02.2013  09:41    <DIR>          v2.0.50727
21.11.2010  08:50    <DIR>          v3.0
21.01.2013  14:21    <DIR>          v3.5
06.05.2013  07:53    <DIR>          v4.0.30319        <---- 2.
               18 Datei(en),        316'600 Bytes
                9 Verzeichnis(se), 396'007'927'808 Bytes frei

C:\Windows\Microsoft.NET\Framework>   <---- 1.
```

Figure 21: Windows XP: Registration of LinUDP.DLL Step 1

Then enter “regasm c:\windows\system32\LinUDP.dll /tlb:LinUDP.tlb /codebase.

```
C:\WINDOWS\system32\cmd.exe

C:\WINDOWS\Microsoft.NET\Framework\v4.0.30319>regasm c:\windows\system32\LinUDP.
dll /tlb:LinUDP.tlb /codebase
Microsoft (R) .NET Framework Assembly Registration Utility 4.0.30319.1
Copyright (C) Microsoft Corporation 1998-2004.  All rights reserved.

Types registered successfully
Assembly exported to 'c:\windows\system32\LinUDP.tlb', and the type library was
registered successfully

C:\WINDOWS\Microsoft.NET\Framework\v4.0.30319>_
```

Figure 22: Windows XP: Registration of LinUDP.DLL Step 2

The registration should be done successful.

To use the Library in Microsoft Excel 2000 or newer, you start your Excel and click on “Extras” (1.), “Macro” (2.) and “Visual Basic Editor” (3.).

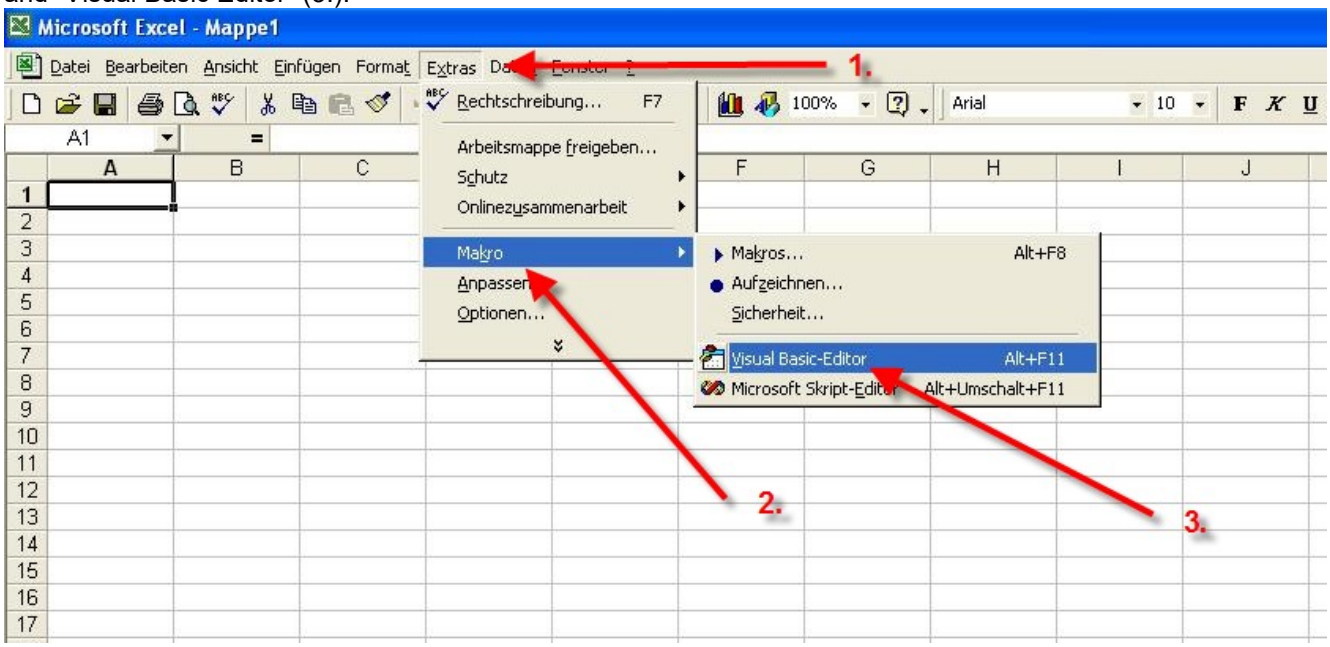


Figure 23: Excel 2010: Open Visual Basic Editor

The Visual Basic Editor appears.

Then click on “Extras” and References (5.)

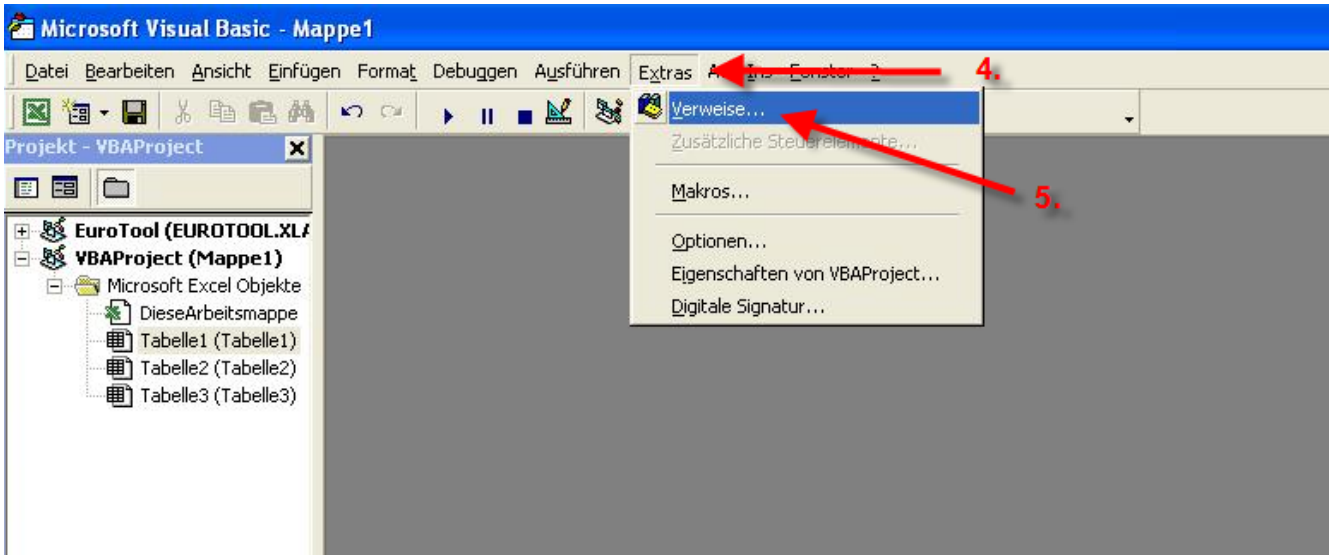


Figure 24: Excel 2010: Open references in Visual Basic

In the references window select the LinUDP driver (6.) and press OK (7.)

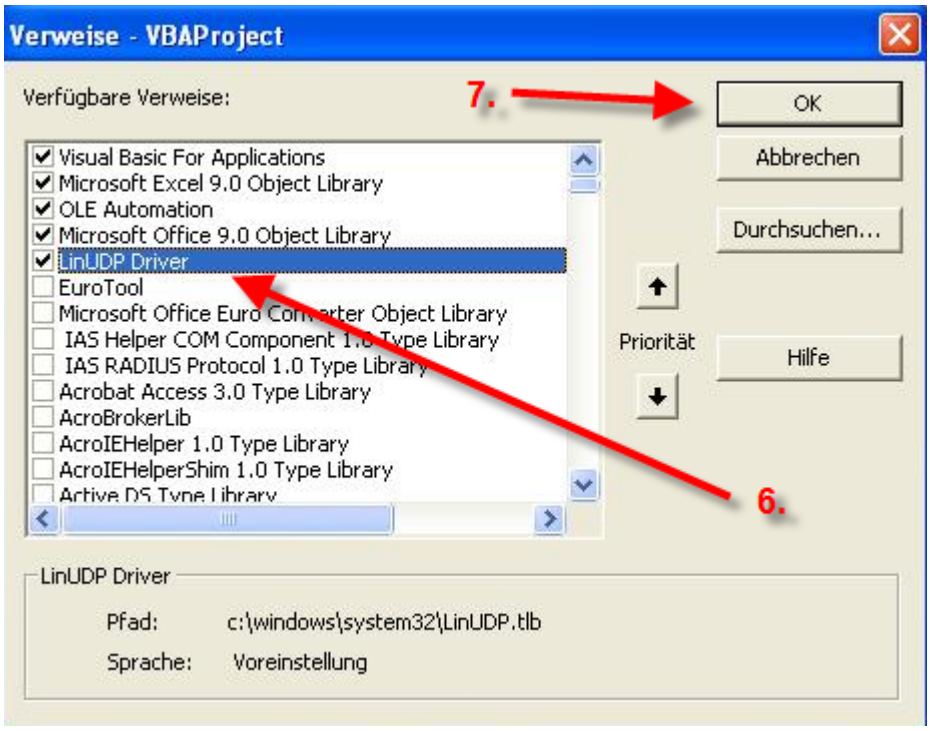


Figure 25: Excel 2010: Add reference for LinUDP

Now you can use the LinUDP driver in Excel!

Contact

SCHWEIZ

NTI AG
Bodenaeckerstr. 2
CH-8957 Spreitenbach

Sales & Administration:	+41-(0)56-419 91 91 office@linmot.com
Tech. Support:	+41-(0)56-544 71 00 support@linmot.com http://www.linmot.com/support
Tech. Support (Skype):	skype:support.linmot
Fax:	+41-(0)56-419 91 92
Web:	http://www.linmot.com/

USA

LinMot USA Inc.
N1922 State Road 120, Unit 1
Lake Geneva, WI 53147
USA

Sales & Administration:	877-546-3270 262-743-2555
Tech. Support:	877-804-0718 262-743-1284
Fax:	800-463-8708 262-723-6688
E-Mail:	usasales@linmot.com
Web:	http://www.linmot-usa.com/

Please visit http://www.linmot.com to find the distribution near you.

Smart solutions are…